Robotics 2

Based on Prof. De Luca's Lectures

April 14, 2026

Contents

1 Kinematic Calibration **3**

2 Kinematic Redundancy **3**
2.1 The Weighted Pseudoinverse 4
2.2 Singularity Robustness: Damped Least Squares 4
2.3 Handling Multiple Tasks and Bounds 4
2.3.1 Deep Dive: The Mathematics of Task Priority 5
2.3.2 Generalization: Recursive Task Priority 6
2.4 Resolution at Acceleration Level 6

3 Kinematic Control (Closing the Loop) **7**
3.1 Nonholonomic Mobile Manipulators (NMM) 7
3.1.1 Separation of Coordinates and Commands 7
3.2 Quadratic Programming (QP) & Inequality Tasks 8
3.2.1 Conflicts, Geometry, and Priorities 8
3.3 Hard Limits & Saturation in the Null Space (SNS) 9
3.4 Inclusion of Hard Cartesian Bounds & Generalized Constraints 11

4 The Dynamic Model **12**
4.1 The Two Problems of Dynamics 12
4.2 The Euler-Lagrange Formulation 12
4.3 Computing Energies: Kinetic and Potential 12
4.4 The Dynamic Model in Matrix Form 13
4.5 Fundamental Properties of the dynamic model 14
4.6 Robot Dynamic Model in Vector Formats 15
4.6.1 Standard Formulation 15
4.6.2 Factorized Formulation 16
4.7 Dynamics of an Actuated Pendulum 16

5 Lagrangian Dynamics **18**
5.1 Actuator Dynamics and Friction 18
5.2 Link Inertia Matrix and Steiner's Theorem 18
5.3 Moving Frames Algorithm for Kinetic Energy 19
5.3.1 Center of Mass Position and Asymmetric Mass Distribution 20
5.3.2 Step-by-Step Example: Dynamic Model of a 2R Planar Robot 20

6 Dynamic model of robots **22**
6.1 Bounds on Dynamic Terms . . . 22
6.2 Adding Real-World Effects: Actuators, Friction, and Elasticity . . . 23
6.3 State Space Representation and Linearization . . . 23
6.4 Coordinate Transformation . . . 24
6.5 Task Space Dynamics with Redundancy . . . 24
6.6 Direct and Inverse Dynamics . . . 24
6.7 Dynamic Scaling of Trajectories . . . 24
6.8 Optimal Point-to-Point Robot Motion . . . 25

7 Dynamic Redundancy Resolution (LQ Optimization) **25**

8 How to Solve Midterm Exercises **27**
8.1 Type A.1: Task Priority and Task Augmentation . . . 27
8.1.1 Example: Iterative Task Priority (3 or more tasks) . . . 27
8.2 Type A.2: Projected and Reduced Gradient . . . 28
8.2.1 Addendum: Gradient for Obstacle Avoidance . . . 29
8.3 Type A.3: Singularity Robustness (DLS) . . . 29
8.4 Type B: SNS Method (Hard Bounds) . . . 30
8.5 Type C.1: Deriving the Dynamic Model . . . 31
8.6 Type C.2: Inverse Dynamics . . . 31
8.6.1 Example: Computing Inverse Dynamics via Regressor . . . 32
8.7 Type D: Skew-Symmetry and Factorizations . . . 33
8.8 Type E: Linear Parameterization and Identification . . . 33
8.9 Type F: Dynamic Redundancy Resolution (LQ Opt.) . . . 34
8.10 Type G: Time Scaling under Torque Bounds . . . 34
8.11 Type H: Energy Dissipation and Optimal Torque Selection . . . 34

9 Proofs **37**
9.1 Minimum Norm Solution for Kinematic Redundancy . . . 37
9.1.1 Proof . . . 37
9.2 Null-Space Projection Matrix (Task Priority) . . . 37
9.2.1 Proof . . . 37
9.3 Linearization for Kinematic Calibration . . . 38
9.3.1 Proof . . . 38
9.4 Symmetry and Positive Definiteness of the Inertia Matrix . . . 38
9.4.1 Proof . . . 38
9.5 Skew-Symmetry of the matrix $\dot{M} - 2C$ . . . 38
9.5.1 Proof . . . 39
9.6 Linearity in the Dynamic Parameters . . . 39
9.6.1 Proof . . . 39

Theory Core Concepts

1 Kinematic Calibration

The Goal: Calibration aims to improve the **absolute accuracy** of a robot via software by correcting the nominal Denavit-Hartenberg (DH) parameters. Note that calibration *cannot* improve **repeatability**, which depends entirely on the mechanical quality of the hardware (e.g., backlash, component stiffness).

Sources of Error: Discrepancies between the real and computed end-effector pose arise from manufacturing tolerances, assembly errors, and factory encoder offsets (which shift the "zero" reference). In an open kinematic chain, these errors amplify as we move towards the end-effector.

The Math & Dimensions: Since direct kinematics is highly non-linear, we linearize it using a first-order Taylor expansion. The resulting basic calibration equation for a single measurement is:

$$\Delta y_e = \Phi(q)\Delta\xi \tag{1}$$

- Δy_e: The measured Cartesian error. It is a 6×1 vector (3 position coordinates + 3 orientation coordinates).
- $\Delta\xi$: The vector of unknown parameter errors. For a robot with n joints, there are 4 DH parameters per joint (α, a, d, θ). Thus, $\Delta\xi$ is a $4n \times 1$ vector.
- $\Phi(q)$: The **Regressor Matrix**. Since it maps a $4n$ vector to a 6 vector, its dimensions are exactly $\mathbf{6 \times 4n}$. It is built using the partial derivatives of the direct kinematics evaluated at the nominal parameters.

The Stacking Procedure (The "Bar" Variables): A single measurement yields at most 6 equations, which is utterly insufficient to find $4n$ unknowns. Therefore, we measure the robot in l different configurations (with $l \gg n$). We "stack" all these measurements into a massive block equation, denoted by the bar notation:

$$\overline{\Delta y_e} = \overline{\Phi}\Delta\xi \tag{2}$$

Now $\overline{\Delta y_e}$ is $6l \times 1$ and $\overline{\Phi}$ is $6l \times 4n$. We have transformed the problem into a heavily overdetermined linear system, which we solve using the pseudoinverse (Least Squares) to find the parameter errors: $\Delta\xi = \overline{\Phi}^{\#}\overline{\Delta y_e}$. This guarantees the minimum norm of the residual estimation error.

2 Kinematic Redundancy

Redundancy is Relative: A robot is never redundant "in absolute terms". It is redundant *with respect to a specific task.* If the number of joints n is strictly greater than the dimension of the task m $(n > m)$, the robot is redundant. Consequently, the differential kinematics $\dot{x} = J(q)\dot{q}$ admits infinite solutions.

To resolve redundancy at the velocity level, we rely on one fundamental equation that dictates the entire theory:

$$\dot{q} = J^{\#}\dot{x} + (I - J^{\#}J)\dot{q}_0 \tag{3}$$

This formula is composed of two fundamental blocks:

1. **The Minimum Norm Term ($J^{\#}\dot{x}$):** Using the Moore-Penrose pseudoinverse $J^{\#} = J^T(JJ^T)^{-1}$, this provides the exact joint velocities strictly necessary to execute the primary task. *What does "Minimum Norm" mean?* It does NOT mean minimum task error. It means that among all infinite ways to move the motors to achieve $\dot{x}$, this solution minimizes the scalar value $\|\dot{q}\|^2 = \dot{q}_1^2 + \cdots + \dot{q}_n^2$. It is the most "lazy" and efficient solution.

2. **The Null-Space Playground ($(I-J^{\#}J)\dot{q}_0$):** The matrix $(I-J^{\#}J)$ is an orthogonal projector. It takes *any* arbitrary velocity vector $\dot{q}_0$ and mathematically "erases" the parts that would disturb the primary task. What remains is a **self-motion** (internal reconfiguration) that leaves the end-effector perfectly still.

2.1 The Weighted Pseudoinverse

The standard pseudoinverse treats all joints equally. The Weighted Pseudoinverse introduces a diagonal matrix $W > 0$, minimizing the weighted sum $\dot{q}^T W \dot{q}$. It is used in 4 specific scenarios:

1. **Mixing Units:** When a robot has both prismatic (m/s) and revolute (rad/s) joints, summing them in a standard norm makes no physical sense. W normalizes the units.

2. **Penalizing Heavy/Slow Joints:** By assigning a high weight to a massive base joint and a low weight to a light wrist, the algorithm will prefer moving the wrist, saving huge amounts of energy.

3. **Joint Limit Avoidance:** By making weights dynamically approach infinity as a joint nears its mechanical limit, the algorithm "understands" that moving that joint has become too costly and halts it smoothly.

4. **Minimizing Actual Kinetic Energy:** If we set $W = M(q)$ (the Inertia Matrix), the algorithm physically minimizes the true instantaneous Kinetic Energy of the robot: $T = \frac{1}{2}\dot{q}^T M(q)\dot{q}$.

2.2 Singularity Robustness: Damped Least Squares

The Problem: The pseudoinverse is computed via Singular Value Decomposition (SVD). It divides by the singular values σ_i of the Jacobian ($1/\sigma_i$). Near a singularity, $\sigma_i \to 0$, causing the required joint velocities to explode to infinity.

The Solution: The Damped Least Squares (DLS, or Levenberg-Marquardt) method abandons the "perfect" minimum norm solution. Instead of ensuring zero task error, it minimizes a mixed objective function: $H = \|\dot{x} - J\dot{q}\|^2 + \mu^2\|\dot{q}\|^2$, where μ^2 is a damping factor. The formula becomes:

$$\dot{q}_{DLS} = J^T(JJ^T + \mu^2 I)^{-1}\dot{x} \tag{4}$$

The term $+\mu^2 I$ guarantees the matrix is always invertible, even in full singularity!

Behavior of the SVD values (Crucial for Exams) The DLS maps the singular values using the function $\frac{\sigma_i}{\sigma_i^2+\mu^2}$. This creates 3 behavioral zones:

- **Far from singularity ($\sigma_i \gg \mu$):** The function acts like $1/\sigma_i$. DLS behaves exactly like the standard pseudoinverse. Perfect tracking.

- **Near singularity ($\sigma_i \approx \mu$):** Instead of going to infinity, the curve peaks at a maximum value of $\frac{1}{2\mu}$. Velocities are capped. The robot safely deviates slightly from the task to protect the hardware.

- **Exactly at singularity ($\sigma_i \to 0$):** The value smoothly drops to 0. The robot halts smoothly along singular directions. Tracking error is high, but the hardware is 100% safe.

2.3 Handling Multiple Tasks and Bounds

When we have multiple Cartesian tasks or hard physical limits, we have four main architectures. Understanding the flaws of the first two is key to understanding why we use the last two.

1. **Task Augmentation (Brute Force):** We stack Task 1 (J_1) and Task 2 (J_2) into a single augmented matrix J_A. We solve with $\dot{q} = J_A^{\#}\dot{x}_A$. *The Flaw:* It treats all tasks equally. If tasks conflict (not enough DOFs), the pseudoinverse yields a Least Squares compromise. The robot will fail *both* tasks.

2. **Extended Jacobian (The Square Way):** We stack tasks until J_{ext} is exactly $n \times n$, then use the standard inverse J_{ext}^{-1}. It guarantees cyclic repeatability. *The Flaw:* Fatal algorithmic singularities. If tasks conflict, the determinant goes to zero and commanded velocities mathematically explode to infinity.

3. **Task Priority (The Safe Hierarchy):** We establish a strict hierarchy. Task 1 is untouchable. Task 2 is executed *strictly* within the Null-Space of Task 1. If they conflict, Task 2 elegantly yields.

4. **SNS - Saturation in the Null Space (Hard Bounds):** SNS applies strict hardware limits (e.g., max motor speed). If a joint exceeds its limit, SNS saturates it at V_{max}, removes that joint from the equation, and uses the Null-Space of the remaining joints to re-route the motion and complete the task safely.

2.3.1 Deep Dive: The Mathematics of Task Priority

Let primary task be $\dot{y}_1 = J_1\dot{q}$ and secondary be $\dot{y}_2 = J_2\dot{q}$. The rigorous Maciejewski-Siciliano formulation is:

$$\dot{q} = J_1^{\#}\dot{y}_1 + (I - J_1^{\#}J_1)\left[J_2(I - J_1^{\#}J_1)\right]^{\#}(\dot{y}_2 - J_2J_1^{\#}\dot{y}_1) \tag{5}$$

Let's break down this complex formula:

- **The Untouchable Block ($J_1^{\#}\dot{y}_1$):** The minimum norm velocity to execute Task 1 perfectly.
- **The Residual Error ($\dot{y}_2 - J_2J_1^{\#}\dot{y}_1$):** While the robot executes Task 1, its arms move. This inevitable motion produces a "free" velocity at the Task 2 level, equal to $J_2(J_1^{\#}\dot{y}_1)$. By subtracting this from the desired $\dot{y}_2$, we get the *residual error*: exactly how much "work" is left to complete Task 2.
- **The Raw Solution ($[J_2 \dots]^{\#} \dots$):** We take the pseudoinverse of the "projected Jacobian" and multiply it by the residual error.

Exam Tip / Oral Question Alert!

The Engineer's Trick (Why the outer projector?)
Notice that the projector $P_1 = (I - J_1^{\#}J_1)$ appears both *inside* the pseudoinverse bracket and *outside* of it. Mathematically, P_1 is **Hermitian** (symmetric) and **Idempotent** ($P_1^2 = P_1$). Because of this, the math says that $(J_2P_1)^{\#} = P_1(J_2P_1)^{\#}$. Therefore, the outer projector is theoretically redundant!
So why write it? *Numerical Robustness.* Computing a pseudoinverse is heavy and generates tiny floating-point approximations in the CPU. If we omit the outer projector, these tiny math errors could "leak" out of the null-space and disturb Task 1. By applying the outer P_1 explicitly at the very end, we act as a final "hardware filter", wiping out any computational garbage and ensuring Task 1 remains mathematically immaculate.

2.3.2 Generalization: Recursive Task Priority

If we have a strict priority hierarchy of s tasks ($1 < 2 < \cdots < s$), writing a single massive formula becomes computationally heavy and unreadable. Instead, we use a **recursive formulation** that updates the solution step-by-step. Let $\dot{y}_k = J_k \dot{q}$ be the k-th task. At step k, the optimal velocity that respects all previous $k-1$ higher-priority tasks is:

$$\dot{q}_k = \dot{q}_{k-1} + (J_k P_{A,k-1})^{\#}(\dot{y}_k - J_k \dot{q}_{k-1}) \tag{6}$$

Where $P_{A,k-1}$ is the projector into the null-space of the *augmented* Jacobian of all previous tasks. The projector itself is updated recursively:

$$P_{A,k} = P_{A,k-1} - (J_k P_{A,k-1})^{\#} J_k P_{A,k-1} \tag{7}$$

starting from $\dot{q}_0 = 0$ and $P_{A,0} = I$. This elegant recursion guarantees that the k-th task will **not perturb** the execution of the previous $k-1$ tasks.

2.4 Resolution at Acceleration Level

Working at the velocity level ($\dot{q}$) is standard, but resolving redundancy at the **acceleration level** ($\ddot{q}$) generates smoother motions and creates a bridge to incorporate the robot's dynamics.

The Mathematical Shift: If the task is defined as $\dot{y} = J(q)\dot{q}$, taking the time derivative yields the task acceleration:

$$\ddot{y} = J(q)\ddot{q} + \dot{J}(q)\dot{q} \tag{8}$$

To isolate the joint accelerations $\ddot{q}$, we rearrange the equation: $J(q)\ddot{q} = \ddot{y} - \dot{J}(q)\dot{q}$.

For convenience, we define the **Equivalent Task Acceleration** as $\ddot{x} \triangleq \ddot{y} - \dot{J}(q)\dot{q}$. The equation simply becomes $J(q)\ddot{q} = \ddot{x}$. This problem is now formally identical to the velocity-level kinematics! The general solution using the null-space projector is:

$$\ddot{q} = J^{\#}(q)\ddot{x} + (I - J^{\#}(q)J(q))\ddot{q}_N \tag{9}$$

Choosing $\ddot{q}_N$:

- **Damping:** A classic choice is to set $\ddot{q}_N = -K_D\dot{q}$ (with $K_D > 0$) to damp and stabilize the internal motions of the joints, which might otherwise drift uncontrollably.
- **Optimization (Discrete Time):** We can choose $\ddot{q}_N$ to locally minimize a generic performance index $G(q, \dot{q})$. Since solving this as a continuous non-linear optimal control problem is extremely hard, we solve it in *discrete time* by assuming the acceleration $\ddot{q}$ is constant over a sampling interval Δt.

Null-space Acceleration by Optimization (Discrete Time) Goal: Find the optimal $\ddot{q}_k$ to decrease $G(q, \dot{q})$ at the next step t_{k+1}. Given the kinematic updates over Δt:

$$\begin{aligned} \dot{q}_{k+1} &= \dot{q}_k + \Delta t \ddot{q}_k \\ q_{k+1} &= q_k + \Delta t \dot{q}_k + \frac{1}{2}\Delta t^2 \ddot{q}_k \end{aligned}$$

We expand G_{k+1} using a first-order Taylor series around the current state $(q_k, \dot{q}_k)$:

$$G_{k+1} \approx G_k + \nabla_q G|_k (q_{k+1} - q_k) + \nabla_{\dot{q}} G|_k (\dot{q}_{k+1} - \dot{q}_k)$$

Substituting the kinematic updates and factoring out $\ddot{q}_k$, we get:

$$G_{k+1} = G_k + \nabla_q G|_k (\Delta t \dot{q}_k) + \Delta t \left(\frac{1}{2}\Delta t \nabla_q G|_k + \nabla_{\dot{q}} G|_k \right) \ddot{q}_k$$

To maximize the decrease of G (Gradient Descent), we must push $\ddot{q}_k$ in the **opposite direction** of the vector multiplying it. Thus, our optimal null-space acceleration is:

$$\ddot{q}_N = -\alpha_k \left(\frac{1}{2} \Delta t \nabla_q G|_k + \nabla_{\dot{q}} G|_k \right) \tag{10}$$

where $\alpha_k > 0$ is the step size.

Exam Tip / Oral Question Alert!

Notice the physical implication of the discrete-time formula above: to influence a cost function acting on acceleration, we must consider both the gradient w.r.t velocity ($\nabla_{\dot{q}} G$) AND the gradient w.r.t position ($\nabla_q G$). However, the position gradient is scaled by $\frac{1}{2}\Delta t$. Therefore, as the sampling time Δt gets smaller (tends to zero), the influence of the position gradient vanishes!

3 Kinematic Control (Closing the Loop)

Up to this point, we have used $\dot{y} = J\dot{q}$ assuming open-loop execution. In reality (or in simulation exercises), using pure kinematic inversion accumulates drift errors due to discrete-time integration or initial mismatches. To guarantee that the task is followed perfectly over time, we must close a feedback loop on the task execution error $e = y_d - y$.

- **At the Velocity Level:** Instead of commanding $\dot{y}_d$ directly, we inject a proportional error correction:

  $$\dot{y}_{cmd} = \dot{y}_d + K_P(y_d - y) \tag{11}$$

  where $K_P > 0$ is a positive definite gain matrix. This commanded velocity is then fed into the pseudoinverse: $\dot{q} = J^{\#}\dot{y}_{cmd}$.

- **At the Acceleration Level:** We replace the feedforward acceleration $\ddot{y}_d$ with a PD-like control law to ensure asymptotic tracking:

  $$\ddot{y}_{cmd} = \ddot{y}_d + K_D(\dot{y}_d - \dot{y}) + K_P(y_d - y) \tag{12}$$

  This commanded acceleration is then used in the acceleration-level redundancy resolution: $\ddot{q} = J^{\#}(\ddot{y}_{cmd} - \dot{J}\dot{q})$.

3.1 Nonholonomic Mobile Manipulators (NMM)

An NMM is a robotic arm mounted on a mobile base (usually wheeled). Solving kinematics for an NMM acts as a bridge between fixed-base robotics and mobile robotics. The key is to separate the system into two sub-components and then fuse them into a single Jacobian framework.

3.1.1 Separation of Coordinates and Commands

- **The Mobile Base:** Described by coordinates q_0 (e.g., x, y, θ). Because it has nonholonomic constraints (a car cannot move sideways), its kinematics are governed by a matrix G: $\dot{q}_0 = G(q_0)u_0$, where u_0 are the available base commands (e.g., linear and angular velocities).

- **The Manipulator Arm:** Described by joint angles q_1. The commands are the joint velocities themselves: $u_1 = \dot{q}_1$.

The total state vector is $q = [q_0^T \quad q_1^T]^T$, and the total available command vector is $u = [u_0^T \quad u_1^T]^T$.

The NMM Jacobian (J_{NMM}) The end-effector velocity $\dot{y}$ depends on both the motion of the base and the motion of the arm:

$$\dot{y} = J_0(q)\dot{q}_0 + J_1(q)\dot{q}_1$$

By substituting the kinematic model of the base ($\dot{q}_0 = G(q_0)u_0$) and the arm ($u_1 = \dot{q}_1$), we obtain:

$$\dot{y} = J_0(q)G(q_0)u_0 + J_1(q)u_1$$

Factoring out the command vector u, we define the **NMM Jacobian**:

$$\dot{y} = \underbrace{\begin{bmatrix} J_0(q)G(q_0) & J_1(q) \end{bmatrix}}_{J_{NMM}(q)} \begin{bmatrix} u_0 \\ u_1 \end{bmatrix} = J_{NMM}(q)u \tag{13}$$

Exam Tip / Oral Question Alert!

The NMM Golden Rule: Once you have formulated J_{NMM}, the problem mathematically reverts to that of a standard fixed-base robot! The system is redundant if the total number of commands (dimension of u) is greater than the task dimension m. You can apply *all* previously learned redundancy resolution techniques (Pseudoinverse, Null-Space, Task Priority, SNS) simply by replacing J with J_{NMM} and $\dot{q}$ with u.

3.2 Quadratic Programming (QP) & Inequality Tasks

While pseudoinverse-based methods handle linear equality constraints beautifully, they cannot natively handle strict hardware limits or bounded zones. **Quadratic Programming (QP)** is the mathematical optimization framework used to handle redundancy when both strict tasks and strict physical limits coexist.

The Concept: The goal of QP in robotics is to minimize a quadratic cost function (usually the kinetic energy or the joint velocity norm $\frac{1}{2}\|\dot{q}\|^2$) subject to two types of constraints:

1. **Equality Constraints ($J\dot{q} = \dot{y}$):** These represent exact operational tasks. Geometrically, in the space of velocity commands, an equality constraint defines a restricted affine space (a line or a hyperplane).
2. **Inequality Constraints ($A\dot{q} \leq b$):** These represent hardware limits (e.g., $|\dot{q}| \leq V_{max}$) or obstacle avoidance thresholds. Geometrically, these define a **convex area** of admissible commands.

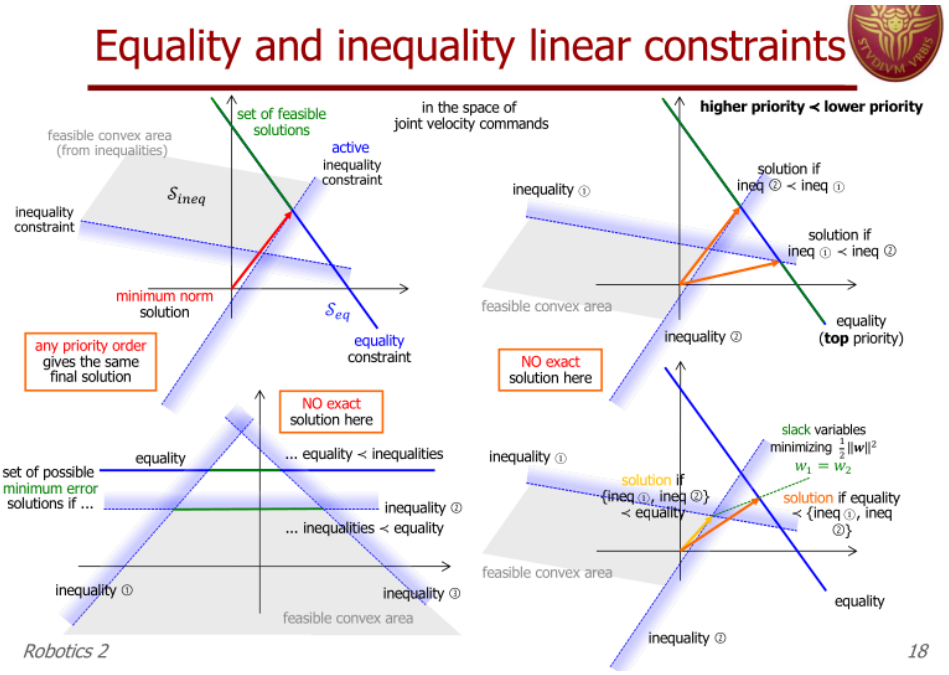
3.2.1 Conflicts, Geometry, and Priorities

- **No Conflict:** If the hyperplane of the equality constraint intersects the convex area of the inequalities, an exact solution exists. The QP solver simply picks the point on that intersection closest to the origin (minimum norm).
- **Conflict (No Exact Solution):** If the equality hyperplane lies completely outside the convex inequality area, the task is physically impossible to execute without breaking a limit.

When a conflict occurs, **Priority** dictates the robot's behavior:

- If the *inequality* has maximum priority (e.g., a hardware motor limit), the solution will stick to the boundary of the convex area. The robot will abandon perfect task execution (it slows down or deviates) to avoid breaking itself.

- If the *equality* has maximum priority, the robot will execute the task but violate the safety limits (a theoretical condition, rarely implemented on real hardware).



Exam Tip / Oral Question Alert!

Slack Variables (w): How does a QP solver mathematically allow a lower-priority task to fail gracefully? It uses **slack variables**. The algorithm rewrites a strict equality $J\dot{q} = \dot{y}$ as $J\dot{q} - w \leq \dot{y}$. The slack w absorbs the error, and the solver attempts to minimize w inside its cost function to deviate from the task as little as possible.
Hierarchical QP (HQP) and Humanoids: Humanoid robots (like HRP-2) are the ultimate use-case for HQP. They must juggle dozens of tasks with strict life-or-death priorities: 1) Do not fall (maintain Center of Mass inside support polygon - strict inequality), 2) Avoid obstacles (strict inequality), 3) Reach for an object (equality task, lower priority).

3.3 Hard Limits & Saturation in the Null Space (SNS)

While Quadratic Programming (QP) is the mathematically optimal framework to handle inequality constraints, solving a full Hierarchical QP at every control cycle requires massive computational power. The **SNS (Saturation in the Null Space)** method is a computationally efficient algorithmic workaround that mimics the safety of QP for box inequalities (like max joint speeds) but operates using the much faster null-space projections.

The Concept: The standard pseudoinverse ignores hardware bounds. If a calculated joint velocity exceeds its physical limit, we cannot simply "chop" it off. Doing so changes the direction of the velocity vector, causing the end-effector to derail from its Cartesian path.

The Mathematical Formulation of SNS To mathematically "turn off" (saturate) a joint without derailing the task, the SNS algorithm uses three key components:

1. The Selection Matrix W: A diagonal matrix used to select which joints are free and which are saturated.

- $W_{jj} = 1$ if joint j is "free" (below its limit).
- $W_{jj} = 0$ if joint j violates its limit and must be saturated.

Multiplying the Jacobian by W creates JW, which effectively ignores the columns of the saturated joints.

2. The Command Components (a and b): When joints are saturated, their motion still affects the end-effector. The remaining free joints must *compensate* for this. We split the velocity command into two parts:

- **The Task Execution Vector (a):** $a = (JW)^{\#}\dot{x}$. This is the primary command required for the free joints to execute the desired task $\dot{x}$.
- **The Compensation Vector (b):** $b = (I - (JW)^{\#}J)\dot{q}_{sat}$. This projects the saturated velocities ($\dot{q}_{sat}$) into the null-space of the free joints. It represents the automatic compensation the free joints *must* execute to ensure the saturated joints don't disturb the task.

3. The Task Scaling Factor (s): What if so many joints reach their limit that the robot physically cannot move the end-effector fast enough to achieve $\dot{x}$? We must slow down the execution of the entire task ($\dot{x}_{new} = s\dot{x}$) to bring velocities back within physical limits. The total commanded velocity is:

$$\dot{q} = s \cdot a + b \tag{14}$$

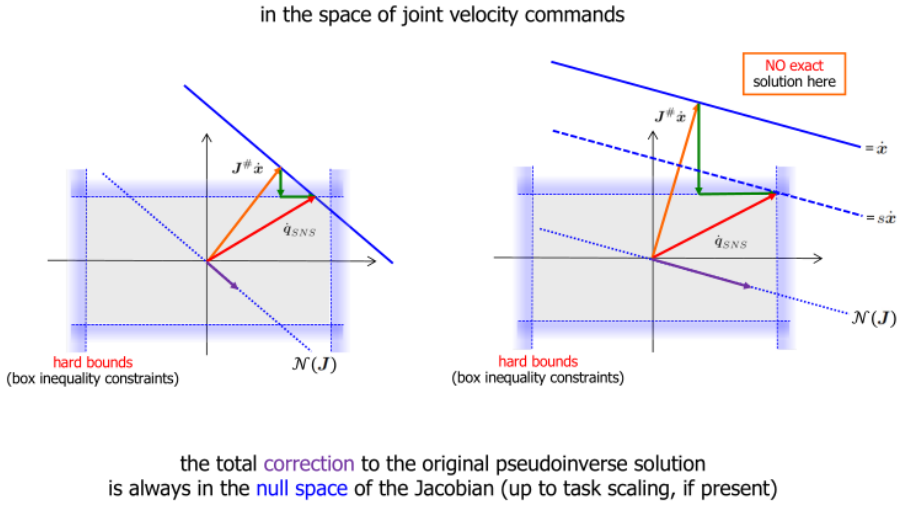
The scale factor $s \in [0, 1]$ is analytically calculated by imposing the joint limits $\dot{q}_{min} \leq \dot{q} \leq \dot{q}_{max}$:

$$\dot{q}_{min,i} \leq s \cdot a_i + b_i \leq \dot{q}_{max,i} \tag{15}$$

From these inequalities, we extract the maximum possible value of s that respects all limits simultaneously. If $s < 1$, the task slows down but does not derail.

The SNS Logic & Algorithm:

1. Calculate the required velocity ignoring constraints ($\dot{q} = J^{\#}\dot{x}$).
2. Check if any joint violates its maximum limit ($|\dot{q}_i| > V_{max}$).
3. If yes, **saturate** the most violating joint, locking its command exactly at V_{max} (with the correct sign), and update the selection matrix W.
4. Recalculate the motion for the remaining "free" joints using components a and b. The null-space projection (b) forces the other joints to compensate for the saturated one, guaranteeing the Cartesian task $\dot{x}$ is still executed perfectly.
5. If limits are still violated, calculate the **Task Scaling Factor** s to slow down the trajectory, or repeat the loop to saturate further joints.



3.4 Inclusion of Hard Cartesian Bounds & Generalized Constraints

While joint limits are simple "box constraints" ($\dot{q}_{min} \leq \dot{q} \leq \dot{q}_{max}$), Cartesian limits (like walls, obstacles, or field-of-view restrictions) are **configuration-dependent**. A joint might be free to rotate, but doing so might push the robot's elbow into a wall. Therefore, these limits must be mapped from the workspace into the joint velocity space.

The Concept: We treat Cartesian boundaries as generalized inequality constraints. When a point on the robot gets too close to a forbidden boundary, we must restrict its velocity *towards* that boundary, while still allowing it to slide parallel to it or move away.

Mathematical Formulation & Calculation of Cartesian Bounds

1. Mapping to Velocity Space: Let a Cartesian constraint be defined by a scalar or vector function $c(q)$ (e.g., the distance between the robot and an obstacle). The physical bound is $c(q) \geq c_{min}$. Differentiating this with respect to time, we map the limit into the velocity domain using the constraint Jacobian J_c:

$$\dot{c} = J_c(q)\dot{q} \tag{16}$$

To ensure the constraint is not violated as the robot approaches the boundary, we enforce an inequality on the velocity:

$$J_c(q)\dot{q} \geq v_{safe} \tag{17}$$

where v_{safe} is typically a function of the distance (e.g., using a potential field or a damping term that forces velocity towards the obstacle to be zero when the distance is critical).

2. Calculating J_c via Control Points (Obstacle Avoidance): The slides generalize these Cartesian bounds by defining r **Control Points** along the robot's structure (e.g., the elbow, the end-effector, or any joint that might collide with the environment). If an obstacle is approaching Control Point r, we only care about the motion of this specific point.

The constraint Jacobian J_c in this scenario is simply the **Geometric Jacobian evaluated at the Control Point** r (denoted as $J_r(q)$). To calculate it:

- Compute the Jacobian using standard forward kinematics formulas, but **stop the calculation at the frame of Control Point** r, completely ignoring the joints and links that come after it.
- If the obstacle only restricts translation (e.g., a wall along the y-axis), you can extract just the relevant linear velocity rows of $J_r(q)$, reducing J_c to a smaller $3 \times n$ (or $1 \times n$) matrix.

The Cartesian velocity of the control point is simply $\dot{x}_r = J_r(q)\dot{q}$.

3. Unified Treatment in Generalized SNS & HQP: By setting Cartesian box inequalities on the control point's velocity ($\dot{x}_{min} \leq J_r(q)\dot{q} \leq \dot{x}_{max}$), the problem is formally converted back into the joint velocity domain.

- **In QP:** These simply become linear inequality constraints of the form $A\dot{q} \leq b$, which the solver handles natively.
- **In Generalized SNS:** The algorithm can now process this exactly as it processes joint limits. However, if a Cartesian constraint becomes active, we cannot simply "turn off" a joint with the matrix W. Instead, J_c is elevated to the **highest priority task** (to strictly enforce $\dot{c}_{safe}$).

4. Sliding on the Boundary (Null-Space Projection): If the robot hits a wall, the velocity vector normal to the wall must be zero. However, the robot can still execute its primary task $\dot{x}$ by exploiting the null-space of the constraint Jacobian. The primary task is projected into the null-space of J_c:

$$P_c = I - J_c^{\#} J_c \tag{18}$$

The allowed motion will be forced to "slide" along the Cartesian constraint, completing the primary task only using the degrees of freedom that do not violate the boundary.

4 The Dynamic Model

The goal of the dynamic model is to find the mathematical relationship $\Phi(q, \dot{q}, \ddot{q}) = u$ that links the joint variables (position, velocity, acceleration) to the generalized torques/forces applied to the joints.

4.1 The Two Problems of Dynamics

Depending on what is known and what needs to be calculated, we distinguish two main problems:

- **Direct Dynamics (Simulation):** Given the initial state $(q(0), \dot{q}(0))$ and the applied inputs $u(t)$, calculate the resulting motion of the robot $q(t)$.
- **Inverse Dynamics (Control):** Given a desired motion trajectory $(q_d(t), \dot{q}_d(t), \ddot{q}_d(t))$, calculate the required torques/forces $u_d(t)$ that the motors must provide to realize it.

4.2 The Euler-Lagrange Formulation

Unlike the Newton-Euler method (which is based on force/moment balancing and is highly recursive), the Euler-Lagrange method is an **energy-based approach**. It is the best method for studying the structural properties of the robot and analyzing control schemes.

Euler-Lagrange Operational Steps

1. **Define the Lagrangian ($\mathcal{L}$):** It is defined as the difference between the total Kinetic Energy (T) and the total Potential Energy (U) of the system:
$$\mathcal{L}(q, \dot{q}) = T(q, \dot{q}) - U(q) \tag{19}$$
2. **Apply the Euler-Lagrange Equation:** For each joint i (from 1 to n), apply the differential operator:
$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} = u_i \tag{20}$$
where u_i is the non-conservative generalized force (e.g., motor torque, friction) performing work on joint i.

4.3 Computing Energies: Kinetic and Potential

Calculating the energies accurately is the core of most dynamic modeling exercises.

1. Kinetic Energy (T): The total kinetic energy is always described by the quadratic form $T = \frac{1}{2}\dot{q}^T M(q)\dot{q}$, where $M(q)$ is the inertia matrix. To compute it step-by-step for a multi-link robot, we apply **König's Theorem** link by link:

$$T = \sum_{i=1}^{n}\left(\frac{1}{2}m_i v_{ci}^T v_{ci} + \frac{1}{2}\omega_i^T I_{ci}\omega_i\right) \tag{21}$$

This theorem simply states that the kinetic energy of a rigid body is the sum of the translational energy of its Center of Mass (CoM) and the rotational energy around its CoM. (v_{ci} is the linear velocity of the CoM, ω_i is the angular velocity, I_{ci} is the barycentric inertia tensor).

Rotational Kinetic Energy and Choice of Reference Frame

In computing the rotational kinetic energy of the i-th link, given by the formula:

$$T_{rot,i} = \frac{1}{2}\boldsymbol{\omega}_i^T \boldsymbol{I}_{c_i}\boldsymbol{\omega}_i \tag{22}$$

it is fundamental that the angular velocity vector $\boldsymbol{\omega}_i$ and the barycentric inertia tensor $\boldsymbol{I}_{c_i}$ are expressed in the **same reference frame** to allow the matrix multiplication.

Practical note: In exercises, it is always convenient to express these quantities in the *link-attached reference frame* (local frame). This way, the inertia matrix $\boldsymbol{I}_{c_i}$ is constant and independent of the robot's configuration.

However, the product $\boldsymbol{\omega}_i^T \boldsymbol{I}_{c_i} \boldsymbol{\omega}_i$ is an **invariant scalar** with respect to the choice of the reference frame. Being proportional to energy (which is an absolute physical quantity), its value does not change if we change our point of view.

Proof of Invariance If we want to transform from a reference frame A to a frame B through a rotation matrix $\boldsymbol{R}$, we know that:

- The angular velocity changes as: $\boldsymbol{\omega}_i^B = \boldsymbol{R}\boldsymbol{\omega}_i^A$
- The inertia tensor changes as: $\boldsymbol{I}_{c_i}^B = \boldsymbol{R}\boldsymbol{I}_{c_i}^A\boldsymbol{R}^T$

Substituting these expressions into the product computed in frame B, we obtain:

$$(\boldsymbol{\omega}_i^B)^T \boldsymbol{I}_{c_i}^B \boldsymbol{\omega}_i^B = (\boldsymbol{R}\boldsymbol{\omega}_i^A)^T (\boldsymbol{R}\boldsymbol{I}_{c_i}^A\boldsymbol{R}^T)(\boldsymbol{R}\boldsymbol{\omega}_i^A) \tag{23}$$

Recalling the transpose property $(\boldsymbol{A}\boldsymbol{B})^T = \boldsymbol{B}^T\boldsymbol{A}^T$:

$$= (\boldsymbol{\omega}_i^A)^T \boldsymbol{R}^T \boldsymbol{R}\boldsymbol{I}_{c_i}^A \boldsymbol{R}^T \boldsymbol{R}\boldsymbol{\omega}_i^A \tag{24}$$

Since the rotation matrix is orthogonal ($\boldsymbol{R}^T\boldsymbol{R} = \boldsymbol{I}$, where $\boldsymbol{I}$ is the identity matrix), the intermediate terms cancel out, yielding:

$$= (\boldsymbol{\omega}_i^A)^T \boldsymbol{I}_{c_i}^A \boldsymbol{\omega}_i^A \tag{25}$$

This demonstrates that the energy value computed in frame B is identical to the one computed in frame A.

2. Potential Energy (U): In standard robotics, we generally only consider the contribution of the gravity field.

$$U = \sum_{i=1}^{n} U_i \quad \text{with} \quad U_i = -m_i g^T r_{c,i} \tag{26}$$

where $r_{c,i}$ is the position vector of the CoM of link i, and g is the gravity acceleration vector (e.g., $g = [0, -9.81, 0]^T$).

Exam Tip / Oral Question Alert!

Watch your Kinematics!
As seen in classic exercises (like the Actuated Pendulum or the PR robot), everything relies on properly defining the reference frames and the CoM positions. If you make a mistake in computing v_{ci} or ω_i (the kinematics), your Kinetic Energy T will be fundamentally flawed, completely compromising the final dynamic equations. *Always double-check your Jacobians and CoM positions first!*

4.4 The Dynamic Model in Matrix Form

By developing the Euler-Lagrange equations, we obtain the standard vector-matrix form of the dynamic model:

$$M(q)\ddot{q} + c(q, \dot{q}) + g(q) = u \tag{27}$$

You must be able to identify and extract the three main blocks:

- **Inertial Terms ($M(q)\ddot{q}$):** $M(q)$ is the $n \times n$ Inertia Matrix. A crucial property is that $M(q)$ is always **symmetric and positive definite**, which guarantees it is always invertible.

- **Coriolis and Centrifugal Terms ($c(q, \dot{q})$):** These terms arise from the kinetic energy derivation and represent the apparent (fictitious) forces acting on the manipulator's links during motion. Since they require non-zero joint velocities ($\dot{q} > 0$), they introduce significant non-linearities and dynamic coupling into the system.

  - *Centrifugal effects:* Proportional to squared joint velocities ($\dot{q}_i^2$). Physically, these represent the radial forces that drive the mass of a link outward from its axis of rotation. The joint actuators must generate counteracting torques to resist these outward pulls and maintain structural rigidity along the desired trajectory.
  - *Coriolis effects:* Proportional to mixed joint velocities ($\dot{q}_i \dot{q}_j$ with $i \neq j$). These effects manifest when a mass moves within a frame of reference that is itself rotating (e.g., the simultaneous rotation of a base joint and the extension of an elbow joint). This combined motion induces transversal forces, causing dynamic cross-coupling where the velocity of one joint directly alters the torque requirements of another.

  While often negligible at low operational speeds—where inertial and gravitational forces dominate—these quadratic velocity terms become highly influential at high speeds. In such regimes, they mandate advanced non-linear control strategies, such as Computed Torque Control (CTC), to actively cancel the dynamic coupling in real-time.

  They can be systematically computed using the **Christoffel Symbols** of the first kind:

$$c_{ij}(q) = \frac{1}{2}\left(\frac{\partial M_i}{\partial q_j} + \left(\frac{\partial M_i}{\partial q_j}\right)^T - \frac{\partial M}{\partial q_i}\right) \tag{28}$$

- **Gravity Terms ($g(q)$):** These are obtained simply by taking the gradient of the total potential energy with respect to the joint coordinates:

$$g(q) = \left(\frac{\partial U}{\partial q}\right)^T \tag{29}$$

4.5 Fundamental Properties of the dynamic model

The dynamic model exhibits structural properties that are fundamental for control theory and stability proofs. Keep these clearly in mind for theoretical questions:

1. **Skew-Symmetry:** The matrix $\dot{M}(q) - 2C(q, \dot{q})$ is always skew-symmetric, provided that the matrix C is chosen using Christoffel symbols. Mathematically, this means:

$$x^T(\dot{M} - 2C)x = 0 \quad \forall x \tag{30}$$

   This property is intimately tied to the conservation of energy and is widely used to prove the asymptotic stability of PD controllers with gravity compensation.

2. **Conservation of Energy:** If there are no dissipative forces (no friction) and no external active forces performing work ($u \equiv 0$), the total energy of the system $E = T + U$ is strictly conserved. Therefore, its time derivative is zero:

$$\dot{E} = 0 \tag{31}$$

3. **Linear Parameterization:** A fundamental structural property of the rigid robot dynamic model is its linearity with respect to a suitable set of constant dynamic parameters (such as masses, lengths, and inertias). The standard Euler-Lagrange equations can be algebraically rewritten as:

$$M(q)\ddot{q} + c(q,\dot{q}) + g(q) = Y(q,\dot{q},\ddot{q})a = \tau \tag{32}$$

where:

- a (often also denoted as π) is a p-dimensional vector of **dynamic coefficients**. These can be the basic parameters (masses, lengths, central inertias) or, more practically, their linear combinations (like the ρ_k coefficients introduced in the 2R planar example).
- $Y(q,\dot{q},\ddot{q})$ is an $n \times p$ matrix known as the **dynamic regressor**. It is a function solely of the joint positions, velocities, and accelerations, meaning it depends entirely on the kinematics of the manipulator.

This linearity property is highly practically relevant, as it forms the mathematical foundation for experimental dynamic parameter identification and the design of adaptive control schemes.

4.6 Robot Dynamic Model in Vector Formats

In the study of manipulator dynamics, the equations of motion can be grouped and written in a compact vector form. In general, the implicit form of the model is:

$$\Phi(q,\dot{q},\ddot{q}) = u \tag{33}$$

4.6.1 Standard Formulation

The first form of the dynamic model highlights the individual contributions and is expressed as:

$$M(q)\ddot{q} + c(q,\dot{q}) + g(q) = u \tag{34}$$

where:

- $M(q)$ is the inertia matrix, which can be extracted from the kinetic energy T using the Hessian.
- $g(q)$ is the vector of gravity terms, calculated as the gradient of the potential energy U:

$$g(q) = \left(\frac{\partial U}{\partial q}\right)^T \tag{35}$$

- $c(q,\dot{q})$ is the vector of Coriolis and centrifugal terms.

Coriolis Vector Analysis The k-th component of the vector $c(q,\dot{q})$ can be rewritten in a quadratic form with respect to the joint velocities:

$$c_k(q,\dot{q}) = \dot{q}^T C_k(q)\dot{q} \tag{36}$$

The associated matrix $C_k(q)$ is defined through the partial derivatives of the inertia matrix:

$$C_k(q) = \frac{1}{2}\left(\frac{\partial M_k}{\partial q} + \left(\frac{\partial M_k}{\partial q}\right)^T - \frac{\partial M}{\partial q_k}\right) \tag{37}$$

where M_k is the k-th column of the matrix $M(q)$. It is important to note that, by construction, the matrix $C_k(q)$ is **symmetric**.

4.6.2 Factorized Formulation

It is often very useful, especially for control design, to factorize the Coriolis term by extracting the velocity vector $\dot{q}$. This yields the second form of the model:

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = u \tag{38}$$

A fundamental note must be made: **the factorization of vector c is not unique**. This means that $c(q,\dot{q}) = N(q,\dot{q})\dot{q}$ holds true for **infinite matrices** N.

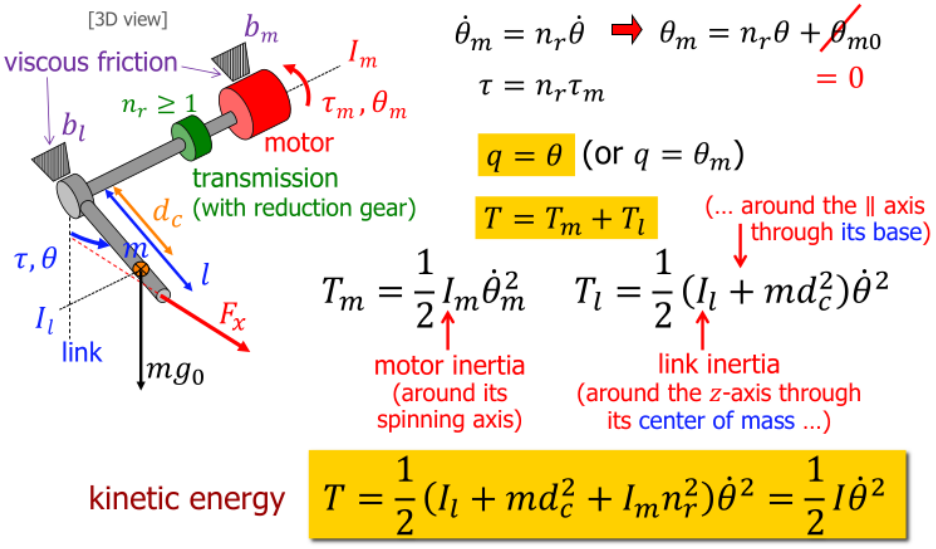
However, the factorization performed with the matrix C is "special", as the elements of this matrix are obtained using **Christoffel symbols**:

$$c_{kj}(q,\dot{q}) = \sum_i c_{kij}(q)\dot{q}_i \tag{39}$$

This specific choice preserves important structural properties of the model (such as the skew-symmetry of the matrix $\dot{M} - 2C$).

4.7 Dynamics of an Actuated Pendulum

In this section, we analyze the dynamic model of an actuated pendulum as derived using the Euler-Lagrange formulation. We will consider the system generalized coordinate as the link angle $q = \theta$. The motor is connected through a reduction gear with transmission ratio $n_r \geq 1$, so the motor angle is $\theta_m = n_r\theta + \theta_{m0}$ (assuming $\theta_{m0} = 0$).



Kinetic Energy and the Inertia Question

The total kinetic energy of the system (T) is the sum of the motor's kinetic energy (T_m) and the link's kinetic energy (T_l):

$$T = T_m + T_l = \frac{1}{2} I_m \dot{\theta}_m^2 + \frac{1}{2}(I_l + md_c^2)\dot{\theta}^2 \tag{40}$$

where:

- I_m is the motor inertia (around its spinning axis).
- I_l is the link inertia around the z-axis through its center of mass.
- m is the link mass, and d_c is the distance from the joint to the center of mass.

The Inertia Question (Parallel Axis / Steiner's Theorem): In the expression for T_l, the term $(I_l + md_c^2)$ represents the equivalent link inertia translated from its center of mass to the parallel axis of rotation at its base. This is a direct application of the parallel axis translation theorem (Steiner's theorem), which states that the moment of inertia around a parallel axis is $I'_{zz} = I_{zz} + mr^2$ (where r is the distance between axes, d_c in this case).

Substituting $\dot{\theta}_m = n_r\dot{\theta}$, we can express the total kinetic energy purely in terms of the link velocity $\dot{\theta}$:

$$T = \frac{1}{2}(I_l + md_c^2 + I_m n_r^2)\dot{\theta}^2 = \frac{1}{2}I\dot{\theta}^2 \tag{41}$$

where $I = I_l + md_c^2 + I_m n_r^2$ is the total equivalent inertia. Notice that the motor inertia I_m is multiplied by n_r^2 when reflected from the motor side to the link side.

Potential Energy and Lagrangian

Assuming gravity $g_0 = 9.81$ m/s^2 acts downwards, the potential energy U depends only on the vertical position of the link's center of mass:

$$U = U_0 - mg_0 d_c \cos\theta \tag{42}$$

The Lagrangian $L = T - U$ is therefore:

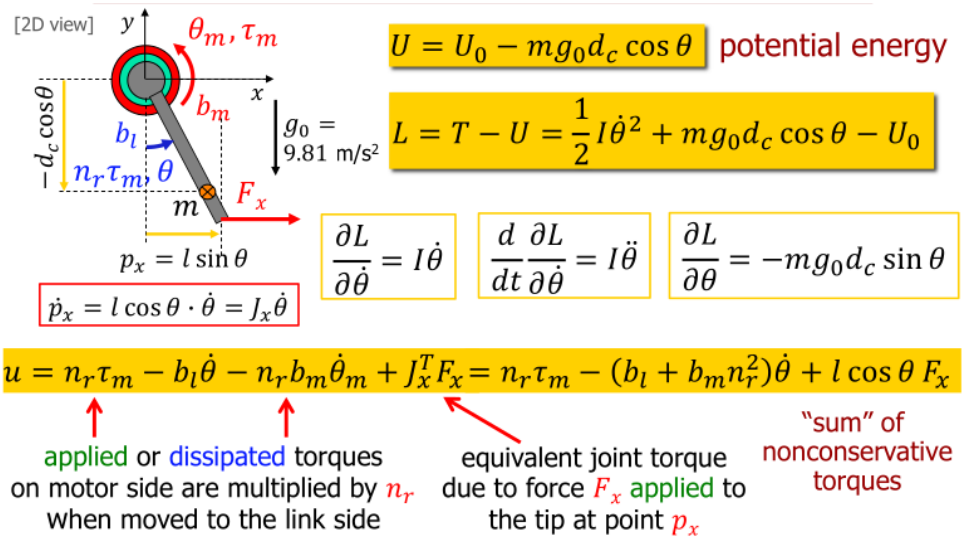
$$L = \frac{1}{2}I\dot{\theta}^2 + mg_0 d_c \cos\theta - U_0 \tag{43}$$

Euler-Lagrange Equations and Generalized Forces

To find the dynamic equation, we compute the required partial derivatives of the Lagrangian:

$$\frac{\partial L}{\partial \dot{\theta}} = I\dot{\theta} \implies \frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}}\right) = I\ddot{\theta} \tag{44}$$

$$\frac{\partial L}{\partial \theta} = -mg_0 d_c \sin\theta \tag{45}$$



The non-conservative generalized forces u performing work on the coordinate θ include the equivalent motor torque τ, viscous friction on both the link (b_l) and motor (b_m), and an external force F_x applied at the tip. The tip position is $p_x = l\sin\theta$, so its velocity is $\dot{p}_x = l\cos\theta\cdot\dot{\theta} = J_x\dot{\theta}$, making the Jacobian $J_x = l\cos\theta$.

Summing these torques:

$$u = n_r\tau_m - b_l\dot{\theta} - n_r b_m\dot{\theta}_m + J_x^T F_x \tag{46}$$

Substituting $\dot{\theta}_m = n_r\dot{\theta}$:

$$u = n_r\tau_m - (b_l + b_m n_r^2)\dot{\theta} + l\cos\theta \cdot F_x \tag{47}$$

Notice how the motor friction b_m is also multiplied by n_r^2 when moved to the link side, similar to the motor inertia.

Final Dynamic Model

Applying the Euler-Lagrange equation $\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{\theta}}\right) - \frac{\partial L}{\partial \theta} = u$, we obtain the dynamic model in terms of the link coordinate $q = \theta$:

$$I\ddot{\theta} + mg_0 d_c \sin\theta = n_r\tau_m - (b_l + b_m n_r^2)\dot{\theta} + l\cos\theta \cdot F_x \tag{48}$$

This scalar equation elegantly captures the inertial, gravitational, and external torque contributions acting on the actuated pendulum system.

Note: The dynamic model can alternatively be expressed in terms of the motor coordinate $q = \theta_m$ by dividing the entire equation by n_r and substituting $\theta = \theta_m/n_r$.

5 Lagrangian Dynamics

The dynamic model of a rigid manipulator is derived from the Lagrangian $\mathcal{L}(q, \dot{q}) = T(q, \dot{q}) - U(q)$. The Euler-Lagrange equations yield:

$$M(q)\ddot{q} + c(q, \dot{q}) + g(q) = \tau \tag{49}$$

5.1 Actuator Dynamics and Friction

In a real manipulator, torques are provided by motors through reduction gears. If n_{ri} is the gear ratio of the i-th joint and I_{mi} is the motor rotor inertia, the kinetic energy of the motors adds a constant diagonal matrix to the inertia matrix:

$$M_{tot}(q) = M(q) + \mathrm{diag}(n_{ri}^2 I_{mi}) \tag{50}$$

Additionally, the real torque τ must compensate for joint friction. The complete model becomes:

$$M_{tot}(q)\ddot{q} + c(q, \dot{q}) + g(q) + F_v\dot{q} + F_c\mathrm{sgn}(\dot{q}) = \tau \tag{51}$$

where F_v is the viscous friction diagonal matrix and F_c is the Coulomb friction diagonal matrix.

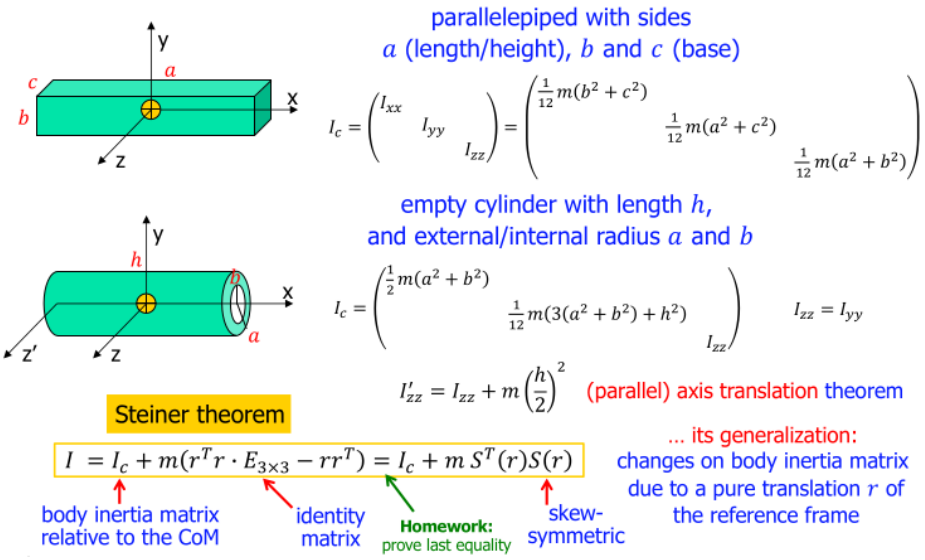
5.2 Link Inertia Matrix and Steiner's Theorem

In the dynamic modeling of a manipulator, the inertia tensor of a rigid link i is typically known with respect to a frame attached to its center of mass (CoM), denoted as I_{ci}. However, for calculating kinetic energy using moving frames, it is often required to express this inertia matrix with respect to a different parallel frame, usually the one attached to the joint origin O_i.

To translate the inertia matrix from the CoM to the joint origin, we apply the **Steiner's Theorem** (also known as the Parallel Axis Theorem). Let m_i be the mass of the link and $r_{ci} = \begin{bmatrix} x_c & y_c & z_c \end{bmatrix}^T$ be the position vector of the CoM relative to O_i. The inertia matrix I_i around the origin O_i is given by:

$$I_i = I_{ci} + m_i\left(\|r_{ci}\|^2 I_{3\times 3} - r_{ci}r_{ci}^T\right) \tag{52}$$

where $I_{3\times3}$ is the 3×3 identity matrix.



Using the skew-symmetric matrix operator $S(\cdot)$ associated with the vector cross product, where $S(r_{ci})v = r_{ci} \times v$, the theorem can be compactly rewritten as:

$$I_i = I_{ci} - m_i S^2(r_{ci}) \tag{53}$$

This transformation is essential because, in the moving frames algorithm, all spatial vectors and inertia tensors must be consistently expressed in their respective local coordinate frames to compute the correct kinetic energy terms.

5.3 Moving Frames Algorithm for Kinetic Energy

For robots with many degrees of freedom (e.g., $n \geq 4$), calculating the inertia matrix and kinetic energy using standard Jacobians becomes symbolically heavy[cite: 3]. It is therefore useful to operate recursively, expressing each vectorial quantity of link i in its own attached "moving frame" (RF_i)[cite: 3]. This approach is particularly convenient when using algebraic or symbolic computation software[cite: 3].

Initialization: Start from the robot base (link 0) at rest: ${}^0\omega_0 = 0$ and ${}^0v_0 = 0$[cite: 6]. Define $\sigma_i = 0$ for a revolute joint and $\sigma_i = 1$ for a prismatic joint[cite: 4].

Forward Recursion (from $i = 1$ to n): For each link i, calculate the kinematic velocities referenced and projected in the local frame i:

1. **Angular velocity of link i:**

$$ {}^i\omega_i = {}^{i-1}R_i^T(q_i) \left[{}^{i-1}\omega_{i-1} + (1-\sigma_i)\dot{q}_i \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right] \tag{54}$$

2. **Linear velocity of the origin O_i:**

$$ {}^i v_i = {}^{i-1}R_i^T(q_i) \left[{}^{i-1}v_{i-1} + \sigma_i \dot{q}_i \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} + {}^{i-1}\omega_{i-1} \times {}^{i-1}r_{i-1,i} \right] \tag{55}$$

3. **Linear velocity of the center of mass (CoM) of link i:**

$$ {}^i v_{ci} = {}^i v_i + {}^i\omega_i \times {}^i r_{ci} \tag{56}$$

(Note: The equations above assume the standard z-axis $[0, 0, 1]^T$ as the joint axis and leverage the already computed vectors from the previous step[cite: 5].)

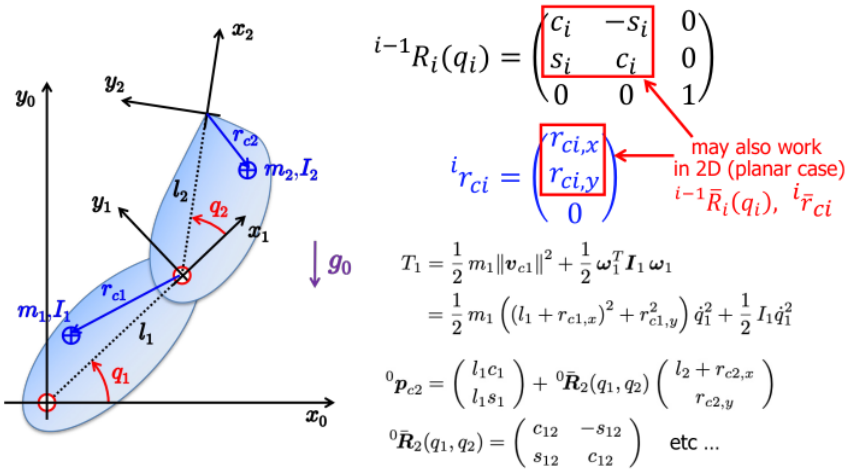
Total Kinetic Energy Calculation: Once all ${}^i v_{ci}$ and ${}^i\omega_i$ are obtained recursively, the total kinetic energy can be written as a sum of local kinetic energies. The main advantage is that the local inertia tensor ${}^i I_i$ is a **constant matrix** in its moving frame:

$$T = \sum_{i=1}^{n} \left(\frac{1}{2} m_i \| {}^i v_{ci} \|^2 + \frac{1}{2} ({}^i\omega_i)^T \, {}^i I_i \, {}^i\omega_i \right) \tag{57}$$

From this expression, matching $T = \frac{1}{2}\dot{q}^T M(q)\dot{q}$, the elements of the inertia matrix $M(q)$ can be extracted to compute the Euler-Lagrange equations.

5.3.1 Center of Mass Position and Asymmetric Mass Distribution

When defining the dynamic parameters of a link, it is crucial to note that the Center of Mass (CoM) is rarely located at its exact geometric center (e.g., $l_i/2$). The position vector r_{ci}, which locates the CoM relative to the local reference frame O_i, depends strictly on the actual mass distribution of the link.



In real robotic manipulators, heavy components such as motors, gearboxes, or counterweights are often positioned as close to the base as possible or extending *behind* the joint axis.

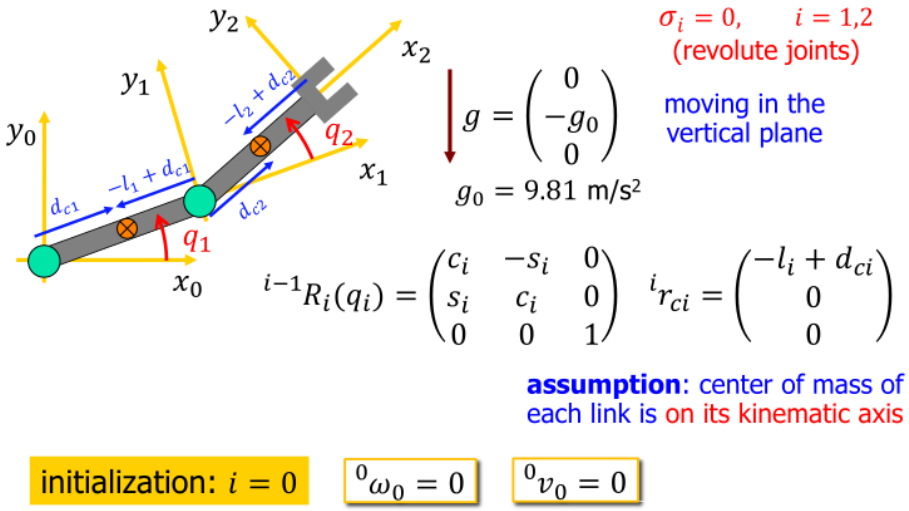
As a result:

- The scalar distance identifying the CoM (often denoted as d_{ci}) can be negative or result in a vector r_{ci} pointing in the negative direction of the link's longitudinal axis (e.g., $r_{c1,x} = -l_1 + d_{c1}$).
- Mechanically, this asymmetry is a deliberate design choice known as **mass balancing** or **counterweighting**.

By shifting the CoM backwards toward the preceding joint, the static moment generated by gravity is significantly reduced. This decreases the required holding torque for the previous actuators, thereby improving the overall payload capacity and energy efficiency of the manipulator.

5.3.2 Step-by-Step Example: Dynamic Model of a 2R Planar Robot

Consider a 2R planar manipulator moving in the vertical plane, with gravity vector $g = \begin{bmatrix} 0 & -g_0 & 0 \end{bmatrix}^T$ where $g_0 = 9.81\, m/s^2$. The assumption is that the center of mass (CoM) of each link lies on its kinematic axis. Let $c_i = \cos(q_i)$, $s_i = \sin(q_i)$, $c_{12} = \cos(q_1 + q_2)$, and $s_{12} = \sin(q_1 + q_2)$.



Initialization (Link 0) The robot base is at rest:

$$ {}^0\omega_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad {}^0v_0 = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{58} $$

First Step (Link 1) Applying the forward recursion for $i = 1$:

$$ {}^1\omega_1 = {}^0R_1^T(q_1) \left[{}^0\omega_0 + \dot{q}_1 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right] = \begin{bmatrix} 0 \\ 0 \\ \dot{q}_1 \end{bmatrix} \tag{59} $$

$$ {}^1v_1 = {}^0R_1^T(q_1) \left[{}^0v_0 + \begin{bmatrix} 0 \\ 0 \\ \dot{q}_1 \end{bmatrix} \times \begin{bmatrix} l_1 c_1 \\ l_1 s_1 \\ 0 \end{bmatrix} \right] = \begin{bmatrix} 0 \\ 0 \\ \dot{q}_1 \end{bmatrix} \times \begin{bmatrix} l_1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ l_1 \dot{q}_1 \\ 0 \end{bmatrix} \tag{60} $$

$$ {}^1v_{c1} = {}^1v_1 + {}^1\omega_1 \times {}^1r_{c1} = \begin{bmatrix} 0 \\ l_1 \dot{q}_1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \dot{q}_1 \end{bmatrix} \times \begin{bmatrix} -l_1 + d_{c1} \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ d_{c1} \dot{q}_1 \\ 0 \end{bmatrix} \tag{61} $$

The kinetic energy of link 1 is:

$$ T_1 = \frac{1}{2} m_1 d_{c1}^2 \dot{q}_1^2 + \frac{1}{2} I_{c1,zz} \dot{q}_1^2 = \frac{1}{2} (I_{c1,zz} + m_1 d_{c1}^2) \dot{q}_1^2 \tag{62} $$

Second Step (Link 2) Applying the forward recursion for $i = 2$:

$$ {}^2\omega_2 = {}^1R_2^T(q_2) \left[{}^1\omega_1 + \dot{q}_2 \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right] = \begin{bmatrix} 0 \\ 0 \\ \dot{q}_1 + \dot{q}_2 \end{bmatrix} \tag{63} $$

$$ {}^2v_2 = {}^1R_2^T(q_2) \left[{}^1v_1 + \begin{bmatrix} 0 \\ 0 \\ \dot{q}_1 \end{bmatrix} \times \begin{bmatrix} l_1 \\ 0 \\ 0 \end{bmatrix} \right] = \begin{bmatrix} l_1 s_2 \dot{q}_1 \\ l_1 c_2 \dot{q}_1 + l_2 (\dot{q}_1 + \dot{q}_2) \\ 0 \end{bmatrix} \tag{64} $$

$$ {}^2v_{c2} = {}^2v_2 + {}^2\omega_2 \times {}^2r_{c2} = \begin{bmatrix} l_1 s_2 \dot{q}_1 \\ l_1 c_2 \dot{q}_1 + d_{c2} (\dot{q}_1 + \dot{q}_2) \\ 0 \end{bmatrix} \tag{65} $$

The kinetic energy of link 2 is:

$$ T_2 = \frac{1}{2} m_2 \left[l_1^2 \dot{q}_1^2 + d_{c2}^2 (\dot{q}_1 + \dot{q}_2)^2 + 2 l_1 d_{c2} c_2 \dot{q}_1 (\dot{q}_1 + \dot{q}_2) \right] + \frac{1}{2} I_{c2,zz} (\dot{q}_1 + \dot{q}_2)^2 \tag{66} $$

Robot Inertia Matrix $M(q)$ To compactly write $T = T_1 + T_2 = \frac{1}{2}\dot{q}^T M(q)\dot{q}$, we introduce dynamic coefficients ρ_k:

$$\begin{aligned}
\rho_1 &= I_{c1,zz} + m_1 d_{c1}^2 + I_{c2,zz} + m_2 d_{c2}^2 + m_2 l_1^2 \\
\rho_2 &= m_2 l_1 d_{c2} \\
\rho_3 &= I_{c2,zz} + m_2 d_{c2}^2 \\
\rho_4 &= g_0(m_1 d_{c1} + m_2 l_1) \\
\rho_5 &= g_0 m_2 d_{c2}
\end{aligned}$$

Extracting the matrix terms yields:

$$m_{11}(q) = \rho_1 + 2\rho_2 \cos(q_2) \tag{67}$$
$$m_{12}(q) = m_{21}(q) = \rho_3 + \rho_2 \cos(q_2) \tag{68}$$
$$m_{22}(q) = \rho_3 \tag{69}$$

Centrifugal and Coriolis Terms $c(q, \dot{q})$ Using the inertia matrix, the Christoffel symbols approach gives:

$$c_1(q, \dot{q}) = -\rho_2 s_2 \dot{q}_2^2 - 2\rho_2 s_2 \dot{q}_1 \dot{q}_2, \qquad c_2(q, \dot{q}) = \rho_2 s_2 \dot{q}_1^2 \tag{70}$$

Gravity Terms $g(q)$ The potential energy is $U = U_1 + U_2 = m_1 g_0 d_{c1} s_1 + m_2 g_0 (l_1 s_1 + d_{c2} s_{12})$. Taking the partial derivatives $g(q) = \partial U / \partial q$:

$$g_1(q) = g_0(m_1 d_{c1} c_1 + m_2 l_1 c_1 + m_2 d_{c2} c_{12}) = \rho_4 c_1 + \rho_5 c_{12} \tag{71}$$
$$g_2(q) = g_0 m_2 d_{c2} c_{12} = \rho_5 c_{12} \tag{72}$$

Final Dynamic Equations Combining all terms into $M(q)\ddot{q} + c(q, \dot{q}) + g(q) = \tau$, the complete model is:

$$\tau_1 = (\rho_1 + 2\rho_2 c_2)\ddot{q}_1 + (\rho_3 + \rho_2 c_2)\ddot{q}_2 - \rho_2 s_2 \dot{q}_2^2 - 2\rho_2 s_2 \dot{q}_1 \dot{q}_2 + \rho_4 c_1 + \rho_5 c_{12} \tag{73}$$
$$\tau_2 = (\rho_3 + \rho_2 c_2)\ddot{q}_1 + \rho_3 \ddot{q}_2 + \rho_2 s_2 \dot{q}_1^2 + \rho_5 c_{12} \tag{74}$$

6 Dynamic model of robots

While the Euler-Lagrange formulation provides the base ideal model $M(q)\ddot{q} + c(q, \dot{q}) + g(q) = \tau$, real-world applications and advanced control schemes require understanding the properties, extensions, and task-space mappings of this model.

6.1 Bounds on Dynamic Terms

For an open-chain (serial) manipulator, the dynamic matrices are physically bounded. There always exist positive real constants k_0 to k_7 such that, for any value of q and $\dot{q}$:

$$k_0 \leq \|M(q)\| \leq k_1 + k_2\|q\| + k_3\|q\|^2 \qquad \text{(Inertia matrix)} \tag{75}$$
$$\|N(q, \dot{q})\| \leq (k_4 + k_5\|q\|)\|\dot{q}\| \qquad \text{(Coriolis/Centrifugal factorization)} \tag{76}$$
$$\|g(q)\| \leq k_6 + k_7\|q\| \qquad \text{(Gravity vector)} \tag{77}$$

Special Case: If the robot has **only revolute joints**, the bounds simplify drastically because sines and cosines are naturally bounded between -1 and 1. The limits become independent of the state q:

$$k_0 \leq \|M(q)\| \leq k_1, \quad \|N(q, \dot{q})\| \leq k_4\|\dot{q}\|, \quad \|g(q)\| \leq k_6 \tag{78}$$

6.2 Adding Real-World Effects: Actuators, Friction, and Elasticity

To make the model usable in practice, we must expand the base equation.

- **Friction:** Modeled primarily as Viscous friction $B_v\dot{q}$ (proportional to velocity) and Coulomb friction $B_c\text{sign}(\dot{q})$ (constant, opposes motion direction).
- **Motor Inertia:** The rotor inertia I_{mi} is amplified by the square of the gear reduction ratio n_{ri}^2 when reflected to the link side.

The complete operational rigid dynamic model becomes:

$$(M(q) + M_m)\ddot{q} + c(q, \dot{q}) + g(q) + B_v\dot{q} + B_c\text{sign}(\dot{q}) = \tau \tag{79}$$

where $M_m = \text{diag}(I_{mi}n_{ri}^2)$.

Including Joint Elasticity: In robots with belts or harmonic drives, joint compliance doubles the generalized coordinates: n link positions (q) and n motor positions (θ). The model splits into a $2n$ system:

$$M(q)\ddot{q} + c(q, \dot{q}) + g(q) + K(q - \theta) = 0 \qquad \text{(Link dynamics)} \tag{80}$$

$$M_m\ddot{\theta} + K(\theta - q) = \tau \qquad \text{(Motor dynamics)} \tag{81}$$

where K is the diagonal matrix of joint stiffness. External torques τ only act directly on the motors.

6.3 State Space Representation and Linearization

For control design and numerical simulation, the second-order model is converted into a **first-order State Space** model by defining the state vector $x = [q^T, \dot{q}^T]^T \in \mathbb{R}^{2n}$:

$$\dot{x} = f(x, u) = \begin{bmatrix} \dot{q} \\ M^{-1}(q)[u - c(q, \dot{q}) - g(q)] \end{bmatrix} \tag{82}$$

where $u = \tau$ is the input vector.

Linearization

To apply linear control techniques, the system is linearized using first-order Taylor expansion around a nominal condition (x_0, u_0), yielding $\delta\dot{x} = A\delta x + B\delta u$, with $A = \frac{\partial f}{\partial x}$ and $B = \frac{\partial f}{\partial u}$.

- **Equilibrium Point (LTI):** If $x_0 = [q_e^T, 0^T]^T$ and $u_0 = g(q_e)$, velocities are zero. Coriolis terms vanish! Matrices A and B are constant:

$$A = \begin{bmatrix} 0 & I \\ -M^{-1}(q_e)K_g & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ M^{-1}(q_e) \end{bmatrix}$$

  where $K_g = \left.\frac{\partial g(q)}{\partial q}\right|_{q=q_e}$ is the **gravity stiffness matrix**.

- **Nominal Trajectory (LTV):** If linearized along a moving reference $x_d(t)$, the Jacobians depend on time: $A(t), B(t)$. Coriolis derivatives must be computed.

6.4 Coordinate Transformation

Given a new set of generalized coordinates $p \in \mathbb{R}^n$ such that $\dot{p} = J(q)\dot{q}$, the dynamic model can be expressed entirely in the new space:

$$M_p(p)\ddot{p} + c_p(p, \dot{p}) + g_p(p) = u_p \tag{83}$$

By applying the principle of virtual work ($u = J^T u_p$) and kinematic differentiation ($\ddot{q} = J^{-1}(\ddot{p} - \dot{J}\dot{q})$), the new terms are:

$$M_p = J^{-T} M J^{-1} \tag{84}$$

$$g_p = J^{-T} g \tag{85}$$

$$c_p = J^{-T} c - M_p \dot{J} J^{-1} \dot{p} \tag{86}$$

Exam Tip / Oral Question Alert!

Even after coordinate transformation, fundamental structural properties hold! M_p remains symmetric and positive definite (outside singularities), and the matrix $(\dot{M}_p - 2C_p)$ remains **skew-symmetric**.

6.5 Task Space Dynamics with Redundancy

For redundant robots ($m < n$), J is not square. We decompose the joint torques into task-producing torques and null-space torques:

$$\tau = J^T F + (I - J^T H)\tau_0 \tag{87}$$

Dynamically Consistent Inverse

To prevent internal null-space motions from generating unwanted accelerations at the end-effector ($\ddot{y}$), we choose H as the transpose of the **inertia-weighted pseudoinverse**:

$$\bar{J} = M^{-1} J^T (J M^{-1} J^T)^{-1} \implies H = \bar{J}^T \tag{88}$$

This perfectly *dynamically decouples* the task dynamics, resulting in:

$$M_y(q)\ddot{y} + n_y(q, \dot{q}) = F \tag{89}$$

where $M_y = (J M^{-1} J^T)^{-1}$ is the **Task Inertia Matrix**.

6.6 Direct and Inverse Dynamics

- **Inverse Dynamics (Control/Planning):** Given desired $q, \dot{q}, \ddot{q}$, find the required joint torques τ. Computed algebraically (no integration required).
- **Direct Dynamics (Simulation):** Given current state $q, \dot{q}$ and applied torques τ, find the resulting acceleration $\ddot{q} = M^{-1}(\tau - c - g)$. Requires numerical integration (e.g., Runge-Kutta) to find future states.

6.7 Dynamic Scaling of Trajectories

If a planned trajectory demands torques exceeding actuator limits ($\tau_{orig}(t) > \tau_{max}$), it is unfeasible. We can recover feasibility by scaling the execution time by a constant factor $k > 1$ (slower execution) without altering the geometric path ($t_{new} = k \cdot t_{old}$).

Scaling Rules & Torque Recovery

Under uniform time scaling by factor k:

- Position q remains unchanged.
- Velocity $\dot{q} \to \dot{q}/k$.
- Acceleration $\ddot{q} \to \ddot{q}/k^2$.

The new required torque scales only in its dynamic components (Inertia and Coriolis). **Gravity does NOT scale**:

$$\tau_{new}(kt) = \frac{1}{k^2}\Big(\tau_{orig}(t) - g(q(t))\Big) + g(q(t)) \tag{90}$$

Exam Tip / Oral Question Alert!

Finding the optimal k^* for feasibility: To recover feasibility for a torque limit τ_{max}, isolate the most critical joint i at the most critical time instant t, and solve:

$$k^* = \max_{i,t} \sqrt{\frac{|\tau_{orig,i}(t) - g_i(q(t))|}{|\tau_{max,i} - g_i(q(t))|}}$$

Mandatory Condition: This is solvable **only if** $\tau_{max,i} > |g_i(q)|\ \forall q$. The actuator must at least be able to support the robot's static weight!

6.8 Optimal Point-to-Point Robot Motion

When moving from point A to B, taking the dynamic model into account allows for trajectory optimization:

- **Minimum-Time Motion:** Bang-bang control. At every instant, at least one actuator is saturated at its physical limits. It is the absolute fastest method but causes severe mechanical stress.
- **Minimum-Energy Motion:** Minimizes the integral of squared torques $\int \tau^2 dt$. Produces smoother, harmonious motions, minimizing mechanical wear at the cost of execution time.

7 Dynamic Redundancy Resolution (LQ Optimization)

Now we make the final leap. We care not only about *how* the robot moves (accelerations), but **how much torque/force** the motors must exert to achieve that motion.

We substitute the joint acceleration $\ddot{q}$ isolated from the dynamic model ($\ddot{q} = M^{-1}(\tau - n(q,\dot{q}))$, where $n = c + g$) into the task kinematic constraint ($J\ddot{q} = \ddot{x}$). This yields a **linear constraint on the torques**:

$$(JM^{-1})\tau = \ddot{x} + JM^{-1}n(q,\dot{q}) \tag{91}$$

Because the robot is redundant, infinite torque vectors τ satisfy this equation. We find the optimal one by formulating a **Linear-Quadratic (LQ) Optimization problem**: Minimize a quadratic cost function $H(\tau) = \frac{1}{2}\tau^T W \tau$ subject to $A\tau = b$. Using Lagrange multipliers, the universal closed-form solution to this mathematical problem is:

$$\tau_{opt} = W^{-1}A^T(AW^{-1}A^T)^{-1}b \tag{92}$$

where in our robotics context: $A = JM^{-1}$ and $b = \ddot{x} + JM^{-1}n$.

By changing the weight matrix W, we obtain the three classic dynamic resolution formulas. Let's see the step-by-step mathematical derivations:

Mathematical Proof of the 3 Closed-Form Dynamic Solutions 1. Minimum Torque Norm ($H_1 = \frac{1}{2}\|\tau\|^2$)

- **Math:** Here there is no weight, so $W = I$ (Identity) and $W^{-1} = I$. Substituting into the general formula:
$$\tau_1 = I \cdot A^T(A \cdot I \cdot A^T)^{-1}b = A^T(AA^T)^{-1}b$$
Notice that $A^T(AA^T)^{-1}$ is the exact definition of the right-pseudoinverse $A^{\#}$.
- **Solution:** $\tau_1 = (JM^{-1})^{\#}(\ddot{x} + JM^{-1}n)$
- **Physics:** Minimizes the raw torque effort. It works well for short trajectories but suffers severely from the **Whipping Effect** on longer paths. Joints accumulate massive internal velocities, causing centrifugal forces to spike, eventually making the required torques explode.

2. Minimum Squared Inverse Inertia Weighted ($H_2 = \frac{1}{2}\tau^T M^{-2}\tau$)

- **Math:** Here $W = M^{-2}$, so $W^{-1} = M^2$. Substituting into the general formula with $A = JM^{-1}$:
$$\tau_2 = M^2(JM^{-1})^T\left[(JM^{-1})M^2(JM^{-1})^T\right]^{-1}b$$
Since inertia matrices are symmetric, $(M^{-1})^T = M^{-1}$. We can simplify:
$$\tau_2 = M^2M^{-1}J^T\left[JM^{-1}M^2M^{-1}J^T\right]^{-1}b$$
$$\tau_2 = MJ^T\left[JIJ^T\right]^{-1}b = M\underbrace{J^T(JJ^T)^{-1}}_{=J^{\#}}b$$
- **Solution:** $\tau_2 = MJ^{\#}(\ddot{x} + JM^{-1}n)$
- **Physics:** Minimizing this specific weighted torque actually corresponds to **minimizing the joint accelerations** $\ddot{q}$. This prevents joints from building up crazy internal velocities, neutralizing the whipping effect. This is the recommended, safest approach for general performance.

3. Minimum Inverse Inertia Weighted ($H_3 = \frac{1}{2}\tau^T M^{-1}\tau$)

- **Math:** Here $W = M^{-1}$, so $W^{-1} = M$. Substituting and simplifying:
$$\tau_3 = M(JM^{-1})^T\left[(JM^{-1})M(JM^{-1})^T\right]^{-1}b$$
$$\tau_3 = MM^{-1}J^T\left[JM^{-1}MM^{-1}J^T\right]^{-1}b$$
The M and M^{-1} terms cancel out perfectly:
$$\tau_3 = J^T\left[JM^{-1}J^T\right]^{-1}b$$
- **Solution:** $\tau_3 = J^T(JM^{-1}J^T)^{-1}(\ddot{x} + JM^{-1}n)$
- **Physics:** This minimizes the instantaneous **kinetic energy** associated with the accelerations. The central term $(JM^{-1}J^T)^{-1}$ is the **Operational Space Inertia Matrix** $\Lambda(q)$. The leading J^T term maps a fictitious Cartesian force directly into joint torques.

Null-Space Damping: To further stabilize these solutions (especially Case 1 to avoid whipping), we can add an arbitrary torque $\tau_0 = -K_D\dot{q}$ projected into the **dynamic null space**. This actively dissipates internal kinetic energy, braking the joints without disturbing the end-effector.

8 How to Solve Midterm Exercises

This section provides step-by-step algorithms to solve the exact typologies of exercises found in the exams.

8.1 Type A.1: Task Priority and Task Augmentation

Algorithm: Solving Task Priority (TP) Problems Goal: Execute a primary task $\dot{x}_1$ perfectly, and a secondary task $\dot{x}_2$ as best as possible using the remaining degrees of freedom.

1. Identify the Jacobians J_1 and J_2.
2. Calculate the pseudoinverse of the primary task: $J_1^{\#} = J_1^T(J_1 J_1^T)^{-1}$.
3. Calculate the Null-Space projector of the primary task: $P_1 = (I - J_1^{\#} J_1)$.
4. The joint velocity for the primary task is: $\dot{q}_1 = J_1^{\#}\dot{x}_1$.
5. Calculate the residual error for the second task: $e_2 = \dot{x}_2 - J_2\dot{q}_1$.
6. Project J_2 into the null space of J_1: $\tilde{J}_2 = J_2 P_1$.
7. The total solution is: $\dot{q}_{TP} = \dot{q}_1 + \tilde{J}_2^{\#} e_2$.

8.1.1 Example: Iterative Task Priority (3 or more tasks)

Algorithm: General Recursive Formulation (Siciliano-Slotine) Goal: Execute multiple tasks strictly ordered by priority (Task 1 > Task 2 > Task 3). This is the exact algorithm to use when handling $N \geq 3$ tasks.

1. Initialization: Start with zero velocity and full available null-space (Identity matrix for an n-DOF robot).

$$P_{A,0} = I_{n\times n}$$
$$\dot{q}_0 = 0$$

2. Task 1 (Primary): Calculate the command for the first task and update the available null-space.

$$P_{1|0} = (J_1 P_{A,0})^{\#} \quad \Big[\text{which simplifies to } J_1^{\#}\Big]$$
$$\dot{q}_1 = \dot{q}_0 + P_{1|0}(\dot{r}_1 - J_1\dot{q}_0) \quad \Big[\text{which simplifies to } J_1^{\#}\dot{r}_1\Big]$$
$$P_{A,1} = P_{A,0} - P_{1|0} J_1 P_{A,0} \quad \Big[\text{which simplifies to } I - J_1^{\#} J_1\Big]$$

3. Task 2 (Secondary): Project J_2 into the null-space left by Task 1.

$$P_{2|1} = (J_2 P_{A,1})^{\#}$$
$$\dot{q}_2 = \dot{q}_1 + P_{2|1}(\dot{r}_2 - J_2\dot{q}_1)$$
$$P_{A,2} = P_{A,1} - P_{2|1} J_2 P_{A,1}$$

4. Task 3 (Tertiary): Project J_3 into the null-space left by Tasks 1 and 2.

$$P_{3|2} = (J_3 P_{A,2})^{\#}$$
$$\dot{q}_{TP} = \dot{q}_2 + P_{3|2}(\dot{r}_3 - J_3\dot{q}_2)$$

Exam Tip / Oral Question Alert!

Pattern Recognition: Notice the repeating pattern? For any task k, you compute the projected inverse $P_{k|k-1}$, update the velocity $\dot{q}_k$ using the error $(\dot{r}_k - J_k\dot{q}_{k-1})$, and finally update the augmented null-space projector $P_{A,k}$. If an exam asks for 4 tasks, you just add one more block following this exact same logic!

Algorithm: Solving Task Augmentation (TA) Problems Goal: Execute multiple tasks simultaneously by grouping them into a single, higher-dimensional augmented task, treating them with equal priority.

1. Identify the Jacobians for the individual tasks: J_1 and J_2.
2. Identify the desired velocities for the tasks: $\dot{x}_1$ and $\dot{x}_2$.
3. Build the *Augmented Jacobian* matrix by stacking the individual Jacobians:
$$J_A = \begin{bmatrix} J_1 \\ J_2 \end{bmatrix}$$
4. Build the *Augmented Task Velocity* vector by stacking the desired velocities:
$$\dot{x}_A = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix}$$
5. Calculate the pseudoinverse of the Augmented Jacobian: $J_A^{\#} = J_A^T(J_A J_A^T)^{-1}$ (assuming it has full row rank, otherwise use the general pseudoinverse).
6. The total joint velocity solution is directly given by: $\dot{q}_{TA} = J_A^{\#}\dot{x}_A$.

Exam Tip / Oral Question Alert!

Task Priority (TP) vs Task Augmentation (TA): In **TP**, Task 1 is guaranteed to be executed perfectly (zero tracking error), while Task 2 is executed only in the null-space of Task 1 (it does not interfere). In **TA**, both tasks are forced into one system with equal weight. If the tasks are kinematically incompatible (i.e., they conflict and the robot doesn't have enough DOFs to do both), the pseudoinverse $J_A^{\#}$ will find a Least Squares solution. This means it will distribute the error between *both* tasks, so neither of them might be perfectly fulfilled!

8.2 Type A.2: Projected and Reduced Gradient

Algorithm: Projected Gradient (PG) Goal: Execute a task $\dot{x}$ while maximizing/minimizing a scalar objective function $H(q)$.

1. Compute the unconstrained solution: $\dot{q}_{task} = J^{\#}\dot{x}$.
2. Compute the gradient of your objective function: $\nabla_q H = \begin{bmatrix} \frac{\partial H}{\partial q_1} & \frac{\partial H}{\partial q_2} & \dots \end{bmatrix}^T$.
3. Define desired null-space velocity: $\dot{q}_0 = \pm\alpha\nabla_q H$ (use $+$ to maximize, $-$ to minimize, with $\alpha > 0$).
4. Calculate Null-Space projector: $P = (I - J^{\#}J)$.
5. Total velocity: $\dot{q}_{PG} = \dot{q}_{task} + P\dot{q}_0$.

Algorithm: Reduced Gradient (RG) Goal: Same as PG, but computationally lighter (no pseudoinverse).

1. Partition the Jacobian into a square non-singular matrix J_a $(m \times m)$ and J_b $(m \times (n-m))$: $J = [J_a \mid J_b]$.
2. Partition joints accordingly: q_a (dependent) and q_b (independent).
3. Compute the *Reduced Gradient*: $\nabla_{q_b} H' = [-(J_a^{-1} J_b)^T \mid I] \nabla_q H$.
4. Update independent joints: $\dot{q}_b = \pm \alpha \nabla_{q_b} H'$.
5. Solve for dependent joints to ensure task execution: $\dot{q}_a = J_a^{-1}(\dot{x} - J_b \dot{q}_b)$.

8.2.1 Addendum: Gradient for Obstacle Avoidance

Algorithm: Maximizing Distance from an Obstacle Goal: Calculate the gradient $\nabla_q H$ rapidly when the objective function $H(q)$ is the distance between the robot and a generic obstacle.

1. Identify the coordinates of the obstacle O_m and the closest point on the robot $P_m(q)$.
2. Calculate the unit vector pointing from the obstacle to the robot: $u = \frac{P_m - O_m}{||P_m - O_m||}$.
3. Compute the Jacobian $J_m(q)$ specifically for the point P_m (NOT the end-effector!).
4. The gradient is simply: $\nabla_q H = J_m^T(q) u$.
5. Use this gradient in the standard Projected Gradient (PG) formula.

Example: Obstacle Avoidance with PG

Problem: The end-effector must execute a velocity task v_e, while the robot must maximize its distance from a circular obstacle centered at O_m. The closest point on the robot is P_m.

1. Compute J_e (Jacobian of the end-effector) and J_m (Jacobian of the point P_m).
2. Calculate the task velocity: $\dot{q}_{task} = J_e^{\#} v_e$.
3. Calculate the distance gradient: $\nabla_q H_{dist} = J_m^T \frac{P_m - O_m}{||P_m - O_m||}$.
4. Define the null-space velocity: $\dot{q}_0 = \alpha \nabla_q H_{dist}$ (use positive α to maximize).
5. Calculate the final command projecting into the null-space of the end-effector:

$$\dot{q} = \dot{q}_{task} + (I - J_e^{\#} J_e) \dot{q}_0$$

8.3 Type A.3: Singularity Robustness (DLS)

Algorithm: Damped Least Squares (DLS) Goal: Find a safe joint velocity $\dot{q}$ to execute a task $\dot{x}$ when the robot is at or very close to a kinematic singularity, preventing the joint velocities from exploding to infinity.

1. Identify the standard task Jacobian J.
2. Choose a damping factor $\lambda > 0$ (often given in the text, or activated only near the singularity).
3. Calculate the Damped Inverse (Levenberg-Marquardt approach):

$$J_{DLS} = J^T (J J^T + \lambda^2 I)^{-1}$$

4. The joint velocity command is: $\dot{q} = J_{DLS}\dot{x}$.

Example: DLS near a Singularity

Problem: A planar robot is in an extended singular configuration. A task velocity $\dot{x}$ is commanded along the singular direction (which is physically impossible). Using standard $J^{\#}$ would result in division by zero. Calculate the DLS velocity.

1. Evaluate J at the singular configuration. The matrix JJ^T will have determinant equal to 0 (it is singular).
2. Compute $(JJ^T + \lambda^2 I)$. The addition of λ^2 on the diagonal shifts the eigenvalues, making the matrix strictly positive definite and therefore invertible.
3. Invert this new regularized matrix.
4. Multiply by J^T to obtain J_{DLS}, and then multiply by the desired task $\dot{x}$:

$$\dot{q} = J^T(JJ^T + \lambda^2 I)^{-1}\dot{x}$$

5. *Conceptual Result:* The robot will execute the feasible components of $\dot{x}$ perfectly, while the impossible (singular) components will result in a bounded tracking error instead of causing joint velocities to explode.

8.4 Type B: SNS Method (Hard Bounds)

Algorithm: Saturation in the Null Space (SNS) Goal: Find joint velocities that realize the task but strictly respect bounds $|\dot{q}_i| \leq V_{max}$.

1. **Initialize:** Selection matrix $W = I$ (all joints free), saturated velocities $\dot{q}_{sat} = 0$.
2. **Compute nominal command:**
   - $a = (JW)^{\#}\dot{x}$ (Task execution term)
   - $b = (I - (JW)^{\#}J)\dot{q}_{sat}$ (Compensation term)
   - $\dot{q} = a + b$ (Initially, $b = 0$ so $\dot{q} = a$)
3. **Check bounds:** Does any element of $\dot{q}$ exceed its maximum? If NO, done. If YES, proceed to 4.
4. **Saturate worst joint:** Take the joint k that most violates the bound.
   - Set $W_{kk} = 0$ (disable joint k).
   - Set the k-th element of $\dot{q}_{sat}$ to $V_{max} \times \text{sign}(\dot{q}_k)$.
5. **Re-calculate a and b:** Using the updated W and $\dot{q}_{sat}$, recalculate $a = (JW)^{\#}\dot{x}$ and $b = (I - (JW)^{\#}J)\dot{q}_{sat}$.
6. **Calculate Task Scaling Factor s:** To ensure free joints don't violate limits while compensating, find the maximum $s \in [0, 1]$ that satisfies for all joints:

$$\dot{q}_{min,i} \leq s \cdot a_i + b_i \leq \dot{q}_{max,i}$$

7. **Final command:** Set $\dot{q} = s \cdot a + b$. If bounds are still violated, loop back to step 4 (saturate the next worst joint).

Exam Tip / Oral Question Alert!

The Magic of a and b: Splitting the command into a (task direction) and b (null-space compensation) transforms the highly nonlinear problem of scaling an inverse kinematics solution into a simple linear inequality $s \cdot a_i + b_i$. This makes finding the optimal scale factor s trivial!

8.5 Type C.1: Deriving the Dynamic Model

Algorithm: Deriving the Inertia Matrix M Goal: Find the symmetric matrix $M(q)$ from kinetic energy $T = \frac{1}{2}\dot{q}^T M(q)\dot{q}$.

1. For each link i, compute the linear velocity of its Center of Mass, v_{ci}, and its angular velocity, ω_i.
2. **Planar Robot Shortcut:** ω_i is simply a scalar on the Z-axis: $\omega_1 = \dot{q}_1$, $\omega_2 = \dot{q}_1 + \dot{q}_2$, etc. $v_{ci} = \frac{d}{dt}p_{ci}(q)$.
3. Write Kinetic Energy for each link: $T_i = \frac{1}{2}m_i\|v_{ci}\|^2 + \frac{1}{2}\omega_i^T I_{ci}\omega_i$.
4. Sum them up: $T_{tot} = \sum T_i$.
5. Expand T_{tot} and group terms into the quadratic form $\frac{1}{2}\sum\sum m_{ij}(q)\dot{q}_i\dot{q}_j$. The coefficients form $M(q)$.

Algorithm: Deriving Coriolis and Centrifugal terms c Method: Christoffel Symbols (Fast Matrix form) Let M_k be the k-th column of $M(q)$. The k-th element of vector c is $c_k = \dot{q}^T C_k(q)\dot{q}$, where:

$$C_k(q) = \frac{1}{2}\left[\frac{\partial M_k}{\partial q} + \left(\frac{\partial M_k}{\partial q}\right)^T - \frac{\partial M}{\partial q_k}\right] \tag{93}$$

Stack the resulting scalars to get the vector $c(q, \dot{q})$.

Algorithm: Deriving Gravity Vector g

1. Find the height y_{ci} (component aligned with gravity vector) of each CoM.
2. Write total potential energy: $U(q) = \sum m_i g_0 y_{ci}$.
3. Differentiate: $g(q) = \left(\frac{\partial U(q)}{\partial q}\right)^T$.

8.6 Type C.2: Inverse Dynamics

Algorithm: Computing Inverse Dynamics Torque Goal: Calculate the required joint torques $\tau_d(t)$ to perfectly track a given desired trajectory $q_d(t)$.

1. **Derive Trajectory:** Given $q_d(t)$, compute its first derivative $\dot{q}_d(t)$ (velocity) and second derivative $\ddot{q}_d(t)$ (acceleration).
2. **Substitute into Model:** Take the symbolic dynamic model you found: $M(q)\ddot{q} + c(q, \dot{q}) + g(q) = \tau$.
3. **Evaluate:** Substitute $q = q_d(t)$, $\dot{q} = \dot{q}_d(t)$, and $\ddot{q} = \ddot{q}_d(t)$ into the equations.
4. **Alternative via Regressor:** If you have already computed the regressor Y and parameters $\mathbf{a}$, you can simply compute $\tau_d(t) = Y(q_d, \dot{q}_d, \ddot{q}_d)\mathbf{a}$. This is often much faster and less prone to algebraic errors!

8.6.1 Example: Computing Inverse Dynamics via Regressor

Problem: Given the linear parameterization $Y(q, \dot{q}, \ddot{q})\mathbf{a} = \tau$ and a desired trajectory $q_d(t) = \begin{bmatrix} A\sin(\omega t) \\ B\cos(\omega t) \end{bmatrix}$, find the required joint torques $\tau_d(t)$.

1. Calculate velocity and acceleration:

$$\dot{q}_d(t) = \begin{bmatrix} A\omega\cos(\omega t) \\ -B\omega\sin(\omega t) \end{bmatrix}$$
$$\ddot{q}_d(t) = \begin{bmatrix} -A\omega^2\sin(\omega t) \\ -B\omega^2\cos(\omega t) \end{bmatrix}$$

2. Evaluate the Regressor Matrix: Substitute $q_d(t)$, $\dot{q}_d(t)$, and $\ddot{q}_d(t)$ into your previously found symbolic matrix $Y(q, \dot{q}, \ddot{q})$ to obtain the time-dependent matrix $Y_d(t)$.

$$Y_d(t) = Y(q_d(t), \dot{q}_d(t), \ddot{q}_d(t))$$

3. Compute the torques: Multiply the evaluated regressor by your vector of constant dynamic parameters $\mathbf{a}$ (which contains grouped terms like m_1, I_1, m_2, I_2).

$$\tau_d(t) = Y_d(t)\mathbf{a}$$

Full Procedure for Linear Parameterization and Inverse Dynamics

Problem: Extract the regressor Y, the parameter vector $\mathbf{a}$, and compute the inverse dynamics torque $\tau_d(t)$ for a given trajectory $q_d(t)$.

1. Expand the Dynamic Model: Write out the full equations of motion for each joint $(\tau_1, \tau_2, \dots)$ by expanding $M(q)\ddot{q} + c(q, \dot{q}) + g(q)$. For example, a generic first joint equation might look like:

$$\tau_1 = (I_1 + m_1 l_{c1}^2 + m_2 l_1^2)\ddot{q}_1 + (I_2 + m_2 l_{c2}^2)(\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_{c2}\cos(q_2)(2\ddot{q}_1 + \ddot{q}_2) + \dots$$

2. Define the Parameter Vector a: Identify the constant physical parameters (masses, inertias, lengths) that always appear grouped together. Define them as a new set of coefficients a_i.

$$a_1 = I_1 + m_1 l_{c1}^2 + m_2 l_1^2$$
$$a_2 = I_2 + m_2 l_{c2}^2$$
$$a_3 = m_2 l_1 l_{c2}$$

So, your parameter vector is $\mathbf{a} = \begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix}^T$.

3. Build the Regressor Matrix $Y(q, \dot{q}, \ddot{q})$: The Regressor Matrix Y is an $n \times p$ matrix (number of joints $\times$ number of dynamic parameters) that contains exclusively the kinematic variables $(q, \dot{q}, \ddot{q})$ and the gravity constant g_0. It separates the motion from the mass/inertia properties.

How to construct it row by row:

- Look at the equation for τ_1 (which corresponds to **Row 1** of Y).
- Ask yourself: *"What multiplies a_1?"* The answer is $\ddot{q}_1$. This goes into position $Y_{1,1}$.
- Ask yourself: *"What multiplies a_2?"* The answer is $(\ddot{q}_1 + \ddot{q}_2)$. This goes into position $Y_{1,2}$.
- Repeat this process for the equation of τ_2 (**Row 2** of Y). If a parameter a_i does not appear in the equation for a specific joint, simply place a 0 in that column (e.g., a_1 does not appear in τ_2, so $Y_{2,1} = 0$).

Applying this logic, we factor out the vector $\mathbf{a}$ and obtain:

$$\begin{bmatrix}\tau_1\\ \tau_2\end{bmatrix} = \underbrace{\begin{bmatrix}\ddot{q}_1 & \ddot{q}_1+\ddot{q}_2 & \cos(q_2)(2\ddot{q}_1+\ddot{q}_2)-\sin(q_2)(2\dot{q}_1\dot{q}_2+\dot{q}_2^2)\\ 0 & \ddot{q}_1+\ddot{q}_2 & \cos(q_2)\ddot{q}_1+\sin(q_2)\dot{q}_1^2\end{bmatrix}}_{Y(q,\dot{q},\ddot{q})} \underbrace{\begin{bmatrix}a_1\\ a_2\\ a_3\end{bmatrix}}_{\mathbf{a}}$$

This 2×3 matrix is your Regressor $Y(q,\dot{q},\ddot{q})$.

4. Compute Inverse Dynamics Torques $\tau_d(t)$: Given a desired trajectory, e.g., $q_d(t) = \begin{bmatrix}A\sin(\omega t)\\ B\cos(\omega t)\end{bmatrix}$:

- Compute velocities $\dot{q}_d(t)$ and accelerations $\ddot{q}_d(t)$ by taking the 1st and 2nd time derivatives of $q_d(t)$.
- Substitute $q_d, \dot{q}_d, \ddot{q}_d$ directly into the matrix Y to obtain the time-dependent matrix $Y_d(t)$.
- Multiply by the constant vector $\mathbf{a}$ to get the required torques.

$$\tau_d(t) = Y_d(t)\mathbf{a}$$

Note: Doing $\tau_d(t) = Y_d(t)\mathbf{a}$ directly outputs the torques without needing to re-evaluate the heavy M, c, and g matrices individually, saving significant time during the exam.

8.7 Type D: Skew-Symmetry and Factorizations

Algorithm: Finding Factorizations S1 and S2 Goal: Find matrices S such that $S\dot{q} = c$, one making $\dot{M} - 2S$ skew-symmetric (S_1), and one not (S_2).

1. **Skew-symmetric S_1:** Stack the Christoffel matrices you found earlier!

$$S_1(q,\dot{q}) = \begin{bmatrix}\dot{q}^T C_1(q)\\ \dot{q}^T C_2(q)\\ \cdots\end{bmatrix} \tag{94}$$

2. **NON skew-symmetric S_2:** Add a dummy matrix $S_0(\dot{q})$ to S_1 such that $S_0(\dot{q})\dot{q} = 0$. For a 2-DOF robot, choose:

$$S_0(\dot{q}) = \begin{bmatrix}0 & -\dot{q}_2\\ \dot{q}_2 & 0\end{bmatrix}$$

Then $S_2 = S_1 + S_0$. Since $S_0\dot{q} = 0$, $S_2\dot{q} = c(q,\dot{q})$, but $\dot{M} - 2S_2$ will NOT be skew-symmetric.

8.8 Type E: Linear Parameterization and Identification

Algorithm: Extracting Regressor Y and parameters a Goal: Rewrite $M\ddot{q} + c + g = \tau$ as $Y(q,\dot{q},\ddot{q})\mathbf{a} = \tau$.

1. Identify all blocks of constants (masses, lengths, inertias). Group terms that always appear together (e.g., $I_1 + m_1 l_{c1}^2$) into a single dynamic coefficient a_k.
2. Define your vector $\mathbf{a} = [a_1, a_2, \ldots, a_p]^T$.
3. Rewrite the equations of motion isolating the variables.
4. Extract the variables into the $n \times p$ matrix Y.

Algorithm: Dynamic Identification Once you have $Y\mathbf{a} = \tau$, to find the true parameters $\mathbf{a}$ experimentally:

1. Sample $q, \dot{q}, \ddot{q}$ and τ at N time instants along an exciting trajectory.
2. Build the stacked Regressor Matrix $\bar{Y}$ ($nN \times p$) and Torque vector $\bar{\tau}$ ($nN \times 1$).
3. Solve using Least Squares: $\hat{\mathbf{a}} = \bar{Y}^{\#}\bar{\tau} = (\bar{Y}^T\bar{Y})^{-1}\bar{Y}^T\bar{\tau}$.

8.9 Type F: Dynamic Redundancy Resolution (LQ Opt.)

Algorithm: Solving LQ Dynamic Redundancy Goal: Find the joint torque τ that executes a Cartesian task $\ddot{y}$ while minimizing a specific physical effort $H(\tau)$.

1. Isolate the "clean" task acceleration: $\ddot{x} = \ddot{y} - \dot{J}\dot{q}$.
2. Group the dynamic bias terms: $n(q, \dot{q}) = c(q, \dot{q}) + g(q)$.
3. Define the constant vector $b = \ddot{x} + JM^{-1}n$.
4. Apply the correct closed-form solution based on the text's request:
   - **Min** $\|\tau\|^2$: $\tau_1 = (JM^{-1})^{\#}b$.
   - **Min** $\|\tau\|^2_{M^{-2}}$: $\tau_2 = MJ^{\#}b$.
   - **Min** $\|\tau\|^2_{M^{-1}}$: $\tau_3 = J^T(JM^{-1}J^T)^{-1}b$.

8.10 Type G: Time Scaling under Torque Bounds

Algorithm: Uniform Time Scaling Goal: Find scale factor $k \geq 1$ to satisfy torque bounds $|u_i| \leq U_{max}$ without altering the geometric path.

1. Determine the max required torque $\tau_{orig}(t)$ during the given trajectory using Inverse Dynamics.
2. Separate the torque into Gravity terms (g) which DO NOT scale, and Inertial/Coriolis terms (τ_{dyn}) which scale with $1/k^2$.
3. Set up the inequality for the most critical joint:
$$\left| \frac{\tau_{dyn}}{k^2} + g \right| \leq U_{max} \tag{95}$$
4. Solve for k:
$$k \geq \sqrt{\frac{|\tau_{dyn}|}{U_{max} - |g|}} \tag{96}$$
5. The new feasible execution time is $T_s = k \cdot T_{old}$.

8.11 Type H: Energy Dissipation and Optimal Torque Selection

Problem: Given the bounds on joint torques $|\tau_j| \leq T_j$, determine the commanded torque τ that maximizes the instantaneous decrease of the total energy $E = T + U$ of the robot.

Algorithm: Maximizing Energy Decrease

Goal: Choose the joint torques τ to make the energy time derivative $\dot{E}$ (or its second derivative $\ddot{E}$ if the robot is completely at rest) as negative as possible.

1. **Case 1: The robot is currently in motion ($\dot{q} \neq 0$)**
   - Start from the power/energy variation formula:
$$\dot{E} = \dot{q}^T\tau = \sum_{j=1}^{n} \dot{q}_j\tau_j \tag{97}$$
   - To make $\dot{E}$ as negative as possible, apply the maximum available torque in the exact *opposite* direction of the current velocity. For each joint j where $\dot{q}_j \neq 0$, set:
$$\tau_{0,j} = -T_j\text{sign}(\dot{q}_j) \tag{98}$$

- If a joint is not moving ($\dot{q}_j = 0$), its torque does not contribute to $\dot{E}$ in this instant, so you can simply command $\tau_{0,j} = 0$.
- The resulting guaranteed maximum energy decrease is:

$$\dot{E}_{max} = -\sum_{\dot{q}_j \neq 0} T_j |\dot{q}_j| < 0 \tag{99}$$

2. **Case 2: The robot is at rest ($\dot{q} = 0$) but not in equilibrium ($g(q_0) \neq 0$)**

- Since $\dot{q} = 0$, the first derivative is strictly zero: $\dot{E} = 0$ regardless of the torque chosen. To induce a decrease in the immediate future, we must force the second derivative $\ddot{E}$ to be negative.
- Differentiate $\dot{E}$: $\ddot{E} = \ddot{q}^T \tau + \dot{q}^T \dot{\tau}$. Since $\dot{q} = 0$, it simplifies to:

$$\ddot{E} = \ddot{q}^T \tau \tag{100}$$

- Use the standard dynamic model $M(q)\ddot{q} + c(q, \dot{q}) + g(q) = \tau$. At $\dot{q} = 0$, the Coriolis terms vanish ($c(q_0, 0) = 0$). Isolate the joint acceleration $\ddot{q}$:

$$\ddot{q} = M^{-1}(q_0)(\tau - g(q_0)) \tag{101}$$

- Substitute $\ddot{q}$ into $\ddot{E}$ to obtain a complete quadratic function of the variable τ:

$$\ddot{E}(\tau) = \tau^T M^{-1}(q_0)\tau - g^T(q_0)M^{-1}(q_0)\tau \tag{102}$$

- Find the theoretical minimum by taking the gradient of $\ddot{E}$ with respect to τ and setting it to zero:

$$\nabla_\tau \ddot{E} = 2M^{-1}(q_0)\tau - M^{-1}(q_0)g(q_0) = \mathbf{0} \tag{103}$$

- Since $M^{-1}(q_0)$ is a positive definite (and therefore invertible) matrix, we can cleanly solve for τ. The unique optimal torque to immediately dissipate energy is exactly half the gravity compensation vector:

$$\tau_0 = \frac{g(q_0)}{2} \tag{104}$$

- *Final Verification:* The Hessian matrix is $\nabla^2_\tau \ddot{E} = 2M^{-1}(q_0) > 0$, guaranteeing this is a global minimum. Evaluating the function yields a strictly negative energy acceleration:

$$\ddot{E}(\tau_0) = -\frac{1}{4} g^T(q_0)M^{-1}(q_0)g(q_0) < 0 \tag{105}$$

Deep Dive: Mathematical Proof for Case 2 ($\dot{q} = 0$)

To rigorously prove why $\tau_0 = \frac{g(q_0)}{2}$ minimizes the energy acceleration $\ddot{E}$, we can analyze the individual contributions of the kinetic energy T and potential energy U.

0. Base Energy Definitions

- **Kinetic Energy:** $T(q, \dot{q}) = \frac{1}{2}\dot{q}^T M(q)\dot{q}$
- **Potential Energy:** $U(q)$, whose gradient with respect to q is the gravity vector: $\frac{\partial U}{\partial q} = g^T(q)$

1. Joint Acceleration Mapping
Starting from the dynamic model at rest ($\dot{q} = 0$, so the Coriolis/centrifugal terms $c(q_0, 0) = \mathbf{0}$):

$$M(q_0)\ddot{q} + g(q_0) = \tau \tag{106}$$

Substituting our optimal torque $\tau = \frac{g(q_0)}{2}$ and solving for $\ddot{q}$:

$$M(q_0)\ddot{q} = -\frac{1}{2}g(q_0) \implies \ddot{q} = -\frac{1}{2}M^{-1}(q_0)g(q_0) \tag{107}$$

2. Kinetic Energy Derivatives (T)

- $T = 0$ (since the robot is at rest, $\dot{q} = 0$).
- $\dot{T} = 0$ (the net power depends linearly on $\dot{q}$, so it vanishes).
- For $\ddot{T}$, evaluating the second time derivative at $\dot{q} = 0$ eliminates all complex velocity-dependent terms, leaving only the acceleration quadratic form:

$$\ddot{T} = \ddot{q}^T M(q_0)\ddot{q} \tag{108}$$

  Substituting $\ddot{q}$ from step 1 (and knowing M^{-1} is symmetric):

$$\ddot{T} = \left(-\frac{1}{2}g^T M^{-1}\right) M \left(-\frac{1}{2}M^{-1}g\right) = \frac{1}{4}g^T M^{-1} g > 0 \tag{109}$$

3. Potential Energy Derivatives (U)

- $U = U(q_0) \geq 0$
- $\dot{U} = g^T(q)\dot{q} = 0$ (since $\dot{q} = 0$).
- For $\ddot{U}$, taking the time derivative of $\dot{U}$ yields $\ddot{U} = \dot{q}^T \frac{\partial g}{\partial q}\dot{q} + g^T\ddot{q}$. At $\dot{q} = 0$, the first term vanishes:

$$\ddot{U} = g^T(q_0)\ddot{q} \tag{110}$$

  Substituting $\ddot{q}$ from step 1:

$$\ddot{U} = g^T(q_0)\left(-\frac{1}{2}M^{-1}(q_0)g(q_0)\right) = -\frac{1}{2}g^T M^{-1} g < 0 \tag{111}$$

4. Total Energy Balance (E)
Summing the derived second derivatives gives the total energy acceleration:

$$\ddot{E} = \ddot{T} + \ddot{U} = \frac{1}{4}g^T M^{-1} g - \frac{1}{2}g^T M^{-1} g = -\frac{1}{4}g^T M^{-1} g < 0 \tag{112}$$

Physical insight: Applying exactly half the required gravity compensation torque allows the robot to "fall" slightly, which immediately decreases its potential energy ($\ddot{U} < 0$) while it starts gaining some kinetic energy ($\ddot{T} > 0$). The mathematical catch is that the potential energy decreases exactly *twice as fast* as the kinetic energy increases, resulting in the maximum possible net instantaneous dissipation of the total energy E.

9 Proofs

9.1 Minimum Norm Solution for Kinematic Redundancy

Theorem: Given a redundant manipulator with differential kinematic equation $\dot{y} = J(q)\dot{q}$, where J has full row rank. Among all possible joint velocity solutions $\dot{q}$ that satisfy the task constraint, the one minimizing the quadratic norm of the joint velocities $\frac{1}{2}\|\dot{q}\|^2$ is given by $\dot{q} = J^{\#}\dot{y}$, where $J^{\#} = J^T(JJ^T)^{-1}$ is the Moore-Penrose right pseudoinverse.

9.1.1 Proof

We can find the solution by formulating a constrained optimization problem using Lagrange multipliers. The Lagrangian function is defined as:

$$L(\dot{q}, \lambda) = \frac{1}{2}\dot{q}^T\dot{q} + \lambda^T(\dot{y} - J\dot{q})$$

where λ is the vector of Lagrange multipliers. We compute the partial derivatives with respect to $\dot{q}$ and set them to zero to find the stationary point:

$$\frac{\partial L}{\partial \dot{q}} = \dot{q}^T - \lambda^T J = 0 \implies \dot{q} = J^T\lambda$$

Substituting this expression for $\dot{q}$ back into the original task constraint $\dot{y} = J\dot{q}$:

$$\dot{y} = J(J^T\lambda) = (JJ^T)\lambda$$

Since J has full row rank, the square matrix (JJ^T) is strictly positive definite and therefore invertible. We can solve for λ:

$$\lambda = (JJ^T)^{-1}\dot{y}$$

Finally, substituting λ into the expression for $\dot{q}$, we obtain the proof:

$$\dot{q} = J^T(JJ^T)^{-1}\dot{y} = J^{\#}\dot{y}$$

9.2 Null-Space Projection Matrix (Task Priority)

Property: The matrix $P(q) = I - J^{\#}(q)J(q)$ is an orthogonal projector that maps any arbitrary joint velocity vector $\dot{q}_0$ into the null-space of the Jacobian J, meaning it does not alter the execution of the primary task.

9.2.1 Proof

To prove that P projects into the null-space of J, we must verify that the contribution of $P\dot{q}_0$ to the task $\dot{y}$ is strictly zero. We multiply the projector by the Jacobian matrix J:

$$JP = J(I - J^{\#}J) = J - JJ^{\#}J$$

Substituting the definition of the right pseudoinverse $J^{\#} = J^T(JJ^T)^{-1}$:

$$JJ^{\#}J = J\left[J^T(JJ^T)^{-1}\right]J = (JJ^T)(JJ^T)^{-1}J = I \cdot J = J$$

Therefore:

$$JP = J - J = 0$$

Furthermore, an orthogonal projector must be idempotent ($P^2 = P$) and symmetric ($P^T = P$). Proving idempotence:

$$P^2 = (I - J^{\#}J)(I - J^{\#}J) = I - 2J^{\#}J + J^{\#}JJ^{\#}J = I - 2J^{\#}J + J^{\#}J = I - J^{\#}J = P$$

9.3 Linearization for Kinematic Calibration

Derivation: To identify the kinematic parameters (e.g., Denavit-Hartenberg parameters π), an iterative least-squares approach is used based on the first-order linearization of the direct kinematics equation.

9.3.1 Proof

Let $p = f(\pi, q)$ be the direct kinematics computing the end-effector pose as a function of the geometric parameters π and the joint variables q. Considering the parameters affected by an error $\Delta\pi = \pi_{real} - \pi_{nom}$, we can expand f in a first-order Taylor series around the nominal parameters π_{nom}:

$$p_{real} \approx f(\pi_{nom}, q) + \left.\frac{\partial f}{\partial \pi}\right|_{\pi=\pi_{nom}} \Delta\pi$$

Defining the measured pose error as $\Delta p = p_{real} - f(\pi_{nom}, q)$, we obtain a linear relationship with respect to the parameter variation:

$$\Delta p \approx J_\pi(q)\Delta\pi$$

where $J_\pi = \frac{\partial f}{\partial \pi}$ is the calibration (or identification) Jacobian. By stacking measurements for m different configurations, an overdetermined linear system is built and solved via pseudoinverse:

$$\Delta\pi = (J^{\#}_{\pi,total})\Delta p_{total}$$

9.4 Symmetry and Positive Definiteness of the Inertia Matrix

Property: In the Lagrangian dynamic model, the robot inertia matrix $M(q)$ is always symmetric and positive definite for any configuration q.

9.4.1 Proof

The total kinetic energy of the manipulator is given by the quadratic form:

$$T = \frac{1}{2}\dot{q}^T M(q)\dot{q}$$

Since a physical system in motion ($\dot{q} \neq 0$) must possess strictly positive kinetic energy ($T > 0$), it directly follows that $M(q)$ is a positive definite matrix. Its symmetry ($M(q) = M^T(q)$) derives from its structural definition via the dyadic expansion of the link Jacobians:

$$M(q) = \sum_{i=1}^{n} \left(m_i J_{v,i}^T J_{v,i} + J_{\omega,i}^T R_i I_i R_i^T J_{\omega,i} \right)$$

where each term in the summation is a symmetric matrix by construction.

9.5 Skew-Symmetry of the matrix $\dot{M} - 2C$

Theorem (Passivity Property): In the dynamic model $M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = u$, the matrix $N(q,\dot{q}) = \dot{M}(q) - 2C(q,\dot{q})$ is skew-symmetric if the elements of C are defined using the Christoffel symbols of the first kind. This implies that for any arbitrary vector x, the quadratic form $x^T(\dot{M} - 2C)x = 0$.

9.5.1 Proof

The generic (i, j) element of the inertia matrix is $m_{ij}(q)$. Its time derivative is:

$$\dot{m}_{ij} = \sum_{k=1}^{n} \frac{\partial m_{ij}}{\partial q_k} \dot{q}_k$$

The elements of the Coriolis and centrifugal matrix C are defined as $c_{ij} = \sum_{k=1}^{n} c_{ijk}\dot{q}_k$, where c_{ijk} are the Christoffel symbols:

$$c_{ijk} = \frac{1}{2}\left(\frac{\partial m_{ij}}{\partial q_k} + \frac{\partial m_{ik}}{\partial q_j} - \frac{\partial m_{jk}}{\partial q_i}\right)$$

Let us analyze the (i, j) element of the matrix $N = \dot{M} - 2C$:

$$n_{ij} = \dot{m}_{ij} - 2c_{ij} = \sum_{k=1}^{n} \left[\frac{\partial m_{ij}}{\partial q_k} - \left(\frac{\partial m_{ij}}{\partial q_k} + \frac{\partial m_{ik}}{\partial q_j} - \frac{\partial m_{jk}}{\partial q_i}\right)\right] \dot{q}_k$$

Canceling out the terms $\frac{\partial m_{ij}}{\partial q_k}$:

$$n_{ij} = \sum_{k=1}^{n} \left(\frac{\partial m_{jk}}{\partial q_i} - \frac{\partial m_{ik}}{\partial q_j}\right) \dot{q}_k$$

Swapping the indices i and j, we get:

$$n_{ji} = \sum_{k=1}^{n} \left(\frac{\partial m_{ik}}{\partial q_j} - \frac{\partial m_{jk}}{\partial q_i}\right) \dot{q}_k = -n_{ij}$$

This proves that the matrix N is skew-symmetric ($N^T = -N$). Consequently, the associated quadratic form is always zero:

$$\frac{1}{2}\dot{q}^T(\dot{M} - 2C)\dot{q} = 0$$

9.6 Linearity in the Dynamic Parameters

Derivation: The rigid-body dynamic equations of a manipulator can be linearly parameterized with respect to a set of base dynamic parameters π (masses, centers of mass, inertia tensor elements).

9.6.1 Proof

The kinetic energy T_i and potential energy U_i of each rigid link are linear functions of the link's 10 standard dynamic parameters (mass m_i, first moments of inertia $m_i c_i$, and second moments of inertia I_i). Since the Lagrangian function $L = T - U$ is a linear combination of these energies, it is also linear in the dynamic parameters. The Euler-Lagrange operator:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}}\right) - \frac{\partial L}{\partial q} = \tau$$

is a linear differential operator. Applying a linear operator to a function that is linear in a set of constant parameters π yields equations of motion that are linearly parameterized. Therefore, the dynamic model can be rearranged as:

$$\tau = Y(q, \dot{q}, \ddot{q})\pi$$

where $Y(q, \dot{q}, \ddot{q})$ is the dynamic regressor matrix, depending only on the kinematics and the motion state.

Student Guidelines

1. Kinematic Calibration

- **Objective:** Estimate parameters (e.g., link lengths) and build the regressor matrix.
- **Where to look in the PDF:**
  - **Section 1:** Basic theory and procedures ("Kinematic Calibration").
  - **Section 9.3:** Details for linearization ("Linearization for Kinematic Calibration").

2. Redundancy Management (Task Priority & Projected Gradient)

- **Objective:** Assign velocity commands for multiple tasks respecting hierarchies, or maximize distance from obstacles.
- **Where to look in the PDF:**
  - **Section 8.1 / 8.1.1:** Practical examples and theory to handle 3 or more hierarchical tasks ("Type A.1: Task Priority...").
  - **Section 8.2 / 8.2.1:** Projected Gradient (PG) method for obstacle avoidance ("Type A.2: Projected and Reduced Gradient").

3. Inverse Kinematics with Hard Bounds (SNS Method)

- **Objective:** Find the minimum-norm commands (usually accelerations) that respect strict joint velocity or acceleration limits.
- **Where to look in the PDF:**
  - **Section 8.4:** Practical guidelines ("Type B: SNS Method - Hard Bounds").
  - **Sections 2.3.2, 2.4, and 3.3:** Basic theory on "Resolution at Acceleration Level" and "Saturation in the Null Space".

4. Kinetic Energy, Inertia Matrices, and Properties

- **Objective:** Calculate the inertia matrix and kinetic energy (including recursive algorithms) or verify the validity of a given matrix.
- **Where to look in the PDF:**
  - **Section 5.3:** Recursive algorithm ("Moving Frames Algorithm for Kinetic Energy").
  - **Sections 4.5 and 9.4:** Validity check for symmetry and positive definiteness of the matrix ("Fundamental Properties..." / "Symmetry and Positive Definiteness...").

5. Deriving the Dynamic Model

- **Objective:** Derive the equations of motion $M(q)\ddot{q} + c(q, \dot{q}) + g(q) = u$ (kinetic energy, potential energy, Euler-Lagrange equations).
- **Where to look in the PDF:**
  - **Section 8.5:** Practical guidelines for deriving the model ("Type C.1: Deriving the Dynamic Model").

6. Factorizations (Skew-Symmetry) and Coriolis Terms

- **Objective:** Derive the Coriolis matrix S or C, or prove the skew-symmetry property for $\dot{M} - 2C$.
- **Where to look in the PDF:**
  - **Section 8.7:** Rules for calculations and proofs ("Type D: Skew-Symmetry and Factorizations").

7. Linear Parameterization and Regressor Matrix

- **Objective:** Extract the base dynamic parameter vector to obtain the form $Y(q, \dot{q}, \ddot{q})a = u$.
- **Where to look in the PDF:**
  - **Section 8.8:** Step-by-step guide to identifying the regressor ("Type E: Linear Parameterization and Identification").

8. Motion Planning and Time Optimization (Time Scaling)

- **Objective:** Recalculate or minimize the execution time of a trajectory while respecting torque/effort limits (Torque bounds).
- **Where to look in the PDF:**
  - **Section 8.10:** Procedures for time-scaling ("Type G: Time Scaling under Torque Bounds").